

1. Dobar stil programiranja

Pre nego što uđete u finese programiranja, mislim da bi trebalo objasniti stilove programiranja.

- Ukoliko stavite ; (tačku-zarez) bilo gde u vašem programu, kompajler će ignorisati sve iza njega sve dok ne dođe do znaka za novi red. Ovo znači da možete staviti komentar unutar programa kako bi vas podsetio šta se uopšte u njemu radi. Ovo je dobra praksa, čak i za najprostije programe. Možda sada u potpunosti razumete šta program radi, ali za par meseci, samo ćete se češkati po glavi. Zbog toga, komentarišite što više. Nema ograničenja.
- Možete pridružiti imena konstantama i registrima (više o ovome kasnije). Dosta je lakše čitati na srpskom u šta upisujete, ili šta označava broj, nego da pokušavate da se setite šta znače svi ti brojevi. Zbog toga, koristite opisna imena kao npr. ZBIR. Primetili ste da sam napisao ime velikim slovima. Ono se tako ističe.
- Napravite neku vrstu zaglavlja u svom programu koristeći tačku-zarez. Primer je dat ispod.

```
.....  
; Autor ;  
; Datum ;  
; Verzija ;  
; Naslov ;  
; ;  
; Opis ;  
; ;  
; ;  
; ;  
; ;  
.....
```

Uočili ste da je napravljena neka vrsta kocke koristeći tačku-zarez. Ovako izgleda lepše.

- Izdvojite instrukcije od početka reda i labela sa par razmaka ili „Tab“ dugmetom. Na taj način assembler prepoznaje da li je reč o instrukciji ili labeli, a i program izgleda preglednije.
- Najzad, isprobajte i dokumentujte vaš program i na papiru. Možete koristiti grafike, algoritme ili šta god želite. To će vam pomoći u pisanju programa, korak po korak.

A sada prelazimo na pravu stvar.

2. Registri

Registar je memorijska lokacija unutar mikrokontrolera u koje se može upisivati, sa koga se može čitati, ili obavljati obe ove funkcije. Najlakše je da ih zamislite kao komad hartije na koji možete upisivati i kasnije čitati razne kratke informacije.

Sledeća skica prikazuje raspored registara unutar PIC16F84 mikrokontrolera. Ne brinite ako nikada ranije niste videli ništa slično ovome. Slika samo prikazuje lokacije (adrese) na kojima se nalaze različiti funkcionalni blokovi (registri) unutar mikrokontrolera, što će pomoći pri razumevanju nekih komandi.

Adresa	BANK0	BANK1	Adresa
00h	INDF ⁽¹⁾	INDF ⁽¹⁾	80h
01h	TMR0	OPTION_REG	81h
02h	PCL	PCL	82h
03h	STATUS	STATUS	83h
04h	FSR	FSR	84h
05h	PORTA	TRISA	85h
06h	PORTB	TRISB	86h
07h	/	/	87h
08h	EEDATA	EECON1	88h
09h	EEADR	EECON2 ⁽¹⁾	89h
0Ah	PCLATH	PCLATH	8Ah
0Bh	INTCON	INTCON	8Bh
0Ch	68 Registra opšte namene (statička RAM memorija)	Mapirani u BANK0	8Ch
.			.
.			.
.			.
.			.
4Fh			CFh
50h	/	/	D0h
.			.
.			.
.			.
7Fh			FFh

/ - Nepostojeća memorijska lokacija (čita se kao 0)

⁽¹⁾ - Nije fizički registar

Prvo što ćete primetiti jeste činjenica da su registri podeljeni u dve grupe, označene sa BANK1 i BANK2. BANK1 se koristi za kontrolisanje trenutnog rada mikrokontrolera, na primer određuje koji pinovi na portu A su ulazni, a koji izlazni. BANK0 se koristi za manipulaciju podacima. Recimo da je potrebno postaviti jedan bit na PORTA na logičku jedinicu (setovati ga). Prvo treba otići u BANK1 i tamo setovati određeni bit, čime se odgovarajući pin na PORTA proglašava za izlazni. Zatim se treba vratiti u BANK0, i poslati logička jedinica (bit 1) na pomenuti pin.

Registri u BANK1 koje će se načešće koristiti su STATUS, TRISA i TRISB. Prvi omogućava prelazak iz BANK0 u BANK1 i obratno, TRISA omogućava definisanje ulaznih i izlaznih bitova PORTA registra, dok TRISB omogućava isto to, ali za PORTB.

Pogledamo izbliza ova 3 registra.

STATUS

Da biste prešli iz BANK0 u BANK1 koristićete STATUS registar. To radite setujući bit 5 STATUS registra. Da bi se vratili u BANK0, resetujete bit 5. STATUS registar se nalazi na adresi 03h (h označava brojeve u heksadecimalnom formatu).

TRISA i TRISB

Ovi registri se nalaze na adresama 85h i 86h, respektivno. Da biste programirali željeni pin tako da bude ulazni ili izlazni, jednostavno pošaljete 1 ili 0 na njegov kontrolni bit u odgovarajućem registru. Pri tome možete koristiti bilo binarni ili heksadecimalni format brojeva. Binarnim se slikovitije predstavlja stanje porta. Ukoliko niste vični pretvaranju binarnih u heksadecimalne brojeve i obratno, koristite bilo koji napredniji kalkulator.

Dakle na PORTA postoji 5 pinova, i samim tim 5 kontrolnih bitova u TRISA registru. Ukoliko biste želeli da definišete neki od ovih pinova kao ulazni, poslaćete "1" na njemu pridruženi bit u TRISA registru. Ukoliko biste želeli da neki od ovih pinova definišete kao izlazni, postavićete njegov bit na "0". Redosled bitova odgovara rasporedu pinova, odnosno bit 0 je RA0, bit 1 je RA1 i tako dalje. Na primer, ukoliko biste želeli da postavite pinove RA0, RA3 i RA4 kao izlazne, a RA1 i RA2 kao ulazne, u registar TRISA ćete poslati ovo: 00000110 (= 06h). Primećujete da se bit 0 koji se odnosi na pin 0 nalazi sa desne strane navedenog niza:

PORTA pin	RA4	RA3	RA2	RA1	RA0
bit broj	4	3	2	1	0
binarna vrednost	0	0	1	1	0

Pinove koji se ne upotrebljavaju u elektronskoj šemi potrebno je definisati kao izlazne. U protivnom varijacije napona na njima prouzrokuju povećanu potrošnju i grejanje mikrokontrolera, a eventualno i njegovu neispravnost.

Isto pravilo važi i za PORTB / TRISB.

PORTA i PORTB

Da biste postavili jedan od izlaznih pinova na visoki logički nivo, jednostavno treba da pošaljete “1” na njemu odgovarajući bit u PORTA ili PORTB registru. Format zapisa je isti kao i za registre TRISA i TRISB. Slično tome, da biste pročitali da li se neki pin (ranije definisan kao ulazni) nalazi na visokom ili niskom logičkom nivou, treba da proverite da li je odgovarajući bit u PORTA ili PORTB registru u setovanom ili resetovanom stanju.

Pre nego što damo primer jednog ulazno-izlaznog assemblerskog koda, treba objasniti funkciju još jednog registra: W.

W

W (eng. **W**orking - radni) registar je registar opšte namene u koji možete smestiti bilo koju brojčanu vrednost koju iz nekog razloga želite privremeno da sačuvate. Nakon što ste pridružili određenu vrednost registru W, možete mu pridružiti neku drugu vrednost (oprez – time biste automatski obrisali prethodnu) ili zapamćenu vrednost iz njega premestiti na neku drugu memorijsku lokaciju (registar). Za razliku od ostalih registara on nema svoju fizičku adresu, već je integrisan u sklopu samog mikrokontrolera. U opisima instrukcija se mogu sresti oznake f, k, b i d. Pri tome je sa **f** (eng. **f**ile) označena adresa jednog od registra iz tabele sa početka ovog poglavlja, **k** označava konstantnu vrednost, **b** je redni broj bita u registru a **d** označava odredište (eng. **d**estination) rezultata operacije.

Primer

Ilustrovaćemo sve do sada naučeno. Nemojte još uvek isprobavati ni kompajlirati kod - to ćete uraditi kada dođete do prvog celovitog programa. Za sada pokušajte da prepoznate način na koji se manipuliše portovima, i pripremite se da naučite par assemblerskih instrukcija.

Za pisanje koda možete koristiti bilo koji tekst editor koji snima čist .txt fajl. Recimo Windowsov Notepad, Notepad++ <http://notepad-plus.sourceforge.net/> ili Linuxov Leafpad <http://tarot.freeshell.org/leafpad/>. Ja uglavnom koristim Code Edit Pro sa <http://www.s12x.com/cep/> zbog naglašavanja instrukcija. Ukoliko se ne možete snaći u njegovom interfejsu, vratite se na Notepad. U jednom od narednih poglavlja, upoznaćete se sa MPLAB razvojnim okruženjem, i njegovim editorom.

Podesićemo PORTA prema prethodnom primeru.

Najpre se prebacite iz BANK0 u BANK1. Ovo se radi setujući bit 5 STATUS registra, čija je adresa 03h.

```
bsf 03h, 5
```

BSF f,b znači “setuj bit b u registru f” (eng. **B**it **S**et **f**). Ovde je f=03h što je adresa STATUS registra i b=5 što označava redni broj bita u STATUS registru. Dakle ovim je rečeno “setuj bit 5 na adresi 03h”.

Sada se, dakle, program nalazi u BANK1.

```
movlw 00110b
```

Ovom instrukcijom smešta se binarna vrednost 00110 (slovo b znači da je broj napisan u binarnom formatu) u W registar. Ovo isto mogli ste da uradite u heksadecimalnom formatu, i tada bi instrukcija izgledala ovako:

```
movlw 06h
```

Obe instrukcije rade. MOVLW k znači “stavi konstantnu vrednost (k) u registar W” (eng. **M**ove **L**iteral to **W**).

Sada treba smestiti ovu vrednost u TRISA registar, da biste podesili PORTA.

```
movwf 85h
```

Instrukcija MOVWF f znači: “preмести vrednost registra W u f registar” (eng. **M**ove **W** to **f**). U ovom slučaju adresa f pokazuje na registar TRISA.

TRISA registar sada sadrži vrednost 00110, ili grafički prikazano:

PORTA pin	RA4	RA3	RA2	RA1	RA0
binarna vrednost	0	0	1	1	0
Ulaz / Izlaz	I	I	U	U	I

Pošto ste podesili pinove na PORTA registru, treba se prebaciti u BANK0 kako biste dalje mogli manipulirati signalima na izlaznim pinovima.

```
bcf 03h, 5
```

Ova instrukcija je suprotna od BSF. BCF f,b znači “resetuj bit b u registru f” (eng. **B**it **C**lear **f**). Brojevi koji slede su adresa registra (f), u ovom slučaju STATUS registra i broj bita (b), u ovom slučaju bit 5. Dakle vi ste u stvari resetovali bit 5 STATUS registra. Posledica poslednje instrukcije je da se sada ponovo nalazite u BANK0.

Ceo prethodni kod zapisan u jednom bloku izgleda ovako:

```
bsf      03h, 5      ; Idi u BANK1
movlw    06h          ; Stavi 00110 u W
movwf    85h          ; Prebaci 00110 u TRISA
bcf      03h, 5      ; Vрати se u BANK0
```

Nakon što snimate fajl, promenite mu txt ekstenziju u asm. Za ovo će vam u Windowsu trebati Total Commander <http://www.ghisler.com/>, a u Linuxu Gnome Commander <http://www.nongnu.org/gcmd/>.

Proučite prethodni primer nekoliko puta, sve dok ne budete u stanju da ga u potpunosti razumete. Za sada ste naučili 4 instrukcije. Još samo 31!

3. Pisanje po portovima

U prethodnom poglavlju ste naučili kako da podesite pinove ulazno-izlaznog porta tako da budu ulazni ili izlazni. U ovom poglavlju ćete naučiti kako da pošaljete neki podatak na portove. U primeru koji sledi cilj će Vam biti da omogućite treptanje jedne LED (eng. **L**ight **E**mitting **D**iode), i u njemu ćete videti jedan potpuni program. Nemojte još uvek isprobavati, kompajlirati ni programirati Vaš PIC listinzima koji slede, jer su oni dati samo kao ilustracija.

Najpre treba podesiti drugi bit PORTA registra tako da bude izlazni:

```
bsf      03h, 5      ; Idi u BANK1
movlw    00h          ; Stavi 00000 u W
movwf    85h          ; Prebaci 00000 u TRISA – svi pinovi su izlazni
bcf      03h, 5      ; Vрати se u BANK0
```

Ovo bi trebalo da Vam bude poznato iz prethodnog primera. Jedina razlika je u tome što ste sada podesili sve pinove PORTA registra kao izlazne, šaljući 00h u TRISA registar.

Zatim treba da uključite LED. To ćete uraditi tako što ćete postaviti jedan od pinova (onaj na kome je LED povezana) u visoko logičko stanje. Drugim rečima poslaćete “1” na odgovarajući pin PIC-a. Evo kako se to radi. (Obratite pažnju na komentar svake linije koda):

```
movlw    04h      ; Upiši 04h u W registar. U binarnom obliku to je 00100
              ; što stavlja “1” na pinu 3, držeći ostale pinove na “0”

movwf    05h      ; Sada prebacite sadržaj iz W (04h) u PORTA,
              ; čija adresa je 05h
```

; Pošto je LED uključena, treba je isključiti.

```
movlw    00h      ; Upiši 0h u W registar. Ovo stavlja 0 na sve pinove

movwf    05h      ; Sada prebacite sadržaj iz W (0h) u PORTA,
              ; čija adresa je 05h
```

Ovim ste postigli da jednom upalite, i zatim ugasite LED. Ono što želite da postignete jeste da LED neprekidno treperi. To možete uraditi vraćajući izvršenje programa na početak. Postavite “labelu” (oznaku) na početak programa, i zatim recite programu da nastavi svoje izvršavanje sa tako označene pozicije.

Labela se postavlja veoma jednostavno: upišite ime, recimo Poc, i zatim prepisite sledeći kod:

```
Poc    movlw      04h    ; Upiši 04h u W registar. U binarnom obliku to je 00100
                                   ; što stavlja "1" na pinu 3, držeći ostale pinove na "0"

        movwf      05h    ; Sada prebacite sadržaj iz W (04h) u PORTA,
                                   ; čija adresa je 05h

        movlw      00h    ; Upiši 0h u W registar. Ovo stavlja 0 na sve pinove

        movwf      05h    ; Sada prebacite sadržaj iz W (0h) u PORTA,
                                   ; čija adresa je 05h

        goto       Poc    ; Idi tamo gde smo rekli Poc
```

Kao što možete videti, prvo je stavljena labela "Poc" na početak programa. Na samom kraju programa smo jednostavno rekli "goto Poc". Instrukcija GOTO k znači "idi na" (eng. **Go**-idi, **To**-na) adresu (k) ili labelu.

Program će sada neprekidno uključivati i isključivati LED od trenutka dovođenja napona napajanja, a prestaće po njegovom isključenju.

Pogledamo još jednom isti program:

```
        bsf        03h, 5
        movlw      00h
        movwf      85h
        bcf        03h, 5
Poc     movlw      04h
        movwf      05h
        movlw      00h
        movwf      05h
        goto       Poc
```

Komentari su namerno izostavljeni i sve što možete videti jesu nizovi instrukcija i brojeva. Ovo može da bude krajnje zbunjujuće ukoliko kasnije pokušavate da ispravite eventualno otkrivenu grešku u programu, ali već i tokom prvobitnog pisanja programa, kada treba zapamtiti sve pojedinačne adrese. Čak i kada biste dodali samo komentare, program bi i dalje delovao prilično nejasno. Zbog toga treba svim brojevima dati imena. Ovo može da se uradi naredbom "equ", koja jednostavno znači "jednako nečemu drugom" (eng. **E**qual – jednak). Važno je da primetite da ovo nije instrukcija za PIC već tzv. pseudo instrukcija koja ima uticaj na sam assembler (više o njemu kasnije). Program koji sadrži "equ" pseudo instrukciju će, dakle, nakon assembliranja u mašinski kod PIC-a, biti identičan programu bez "equ" naredbe. Ovom pseudo instrukcijom pridružujete ime nekom određenom broju.

Postupite ovako: izbacite sve komentare ali imenujte registre u Vašem programu, a zatim ocenite čitljivost takvog programa.

```
STATUS      equ   03h   ; Ovo pridružuje reč STATUS vrednosti 03h, što je
                        ; adresa STATUS registra

TRISA       equ   85h   ; Ovo pridružuje reč TRISA vrednosti 85h, što je adresa
                        ; TRISA registra

PORTA       equ   05h   ; Ovo pridružuje reč PORTA vrednosti 05h, što je adresa
                        ; PORTA registra

RP0         equ   00100000b ; Ovo pridružuje reč RP0 vrednosti 05h, što je
                        ; oznaka petog bita STATUS registra
```

Imena moraju biti definisana pre nego što budu upotrebljena, i zbog toga ih uvek definišite na samom početku svakog programa. Nakon imenovanja registara, unesite njihova imena i u aktivni deo programa. Ukoliko sada prepisete program bez komentara, ali sa imenovanim registrima, moćićete da uporedite preglednost i čitljivost takvog listinga sa prethodnim:

```
STATUS      equ 03h
TRISA       equ 85h
PORTA       equ 05h
RP0         equ 00100000b

                bsf      STATUS, RP0
                movlw    00h
                movwf    TRISA
                bcf      STATUS, RP0

Poc
                movlw    04h
                movwf    PORTA
                movlw    00h
                movwf    PORTA
                goto     Poc
```

Sigurno ste primetili da imenovani registri čine praćenje programa znatno lakšim, čak i bez ikakvih komentara. Takođe možete primetiti da se imenovani registri razlikuju od labele po tome što su im sva slova velika. Na taj način ne možete doći u situaciju da kasnije razmišljate da li je neka takva oznaka labela ili imenovani registar.

Možda vam deluje čudno što PORTA registar i RP0 bit STATUS registra imaju istu vrednost (05h). Međutim, razlika je očigledna. POTRA predstavlja adresu registra (05h), a RP0 bit 5 STATUS registra.

Takođe možete videti da su instrukcije odvojene od početka reda. To je obavezno, da ih assembler ne bi preveo kao labele.

4. Prazne petlje

U prethodnom programu postoji mala greška. Skoro svaka instrukcija zahteva jedan instrukcijski ciklus da bi se izvršila. Instrukcijski ciklus je 4 puta veći od takta oscilatora. Ukoliko kao oscilator za takt za PIC16F84 koristite kvarni kristal od 4MHz trajanje svake instrukcije će biti 4MHz/4ciklusa, ili 1μS. Kako se koristi samo 5 instrukcija, LED će se upaliti i ugaziti u samo 6μS. Zbog čega 6 a ne 5? Zbog toga što instrukcije koje menjaju stanje programskog brojača (eng. Program Counter – više o njemu kasnije) poput GOTO za svoje izvršenje troše 2 vremenska ciklusa. Kako je to treptanje previše brzo da biste ga uopšte mogli primetiti, delovaće Vam da LED svetli konstantno, ali sa pola snage. Dakle treba napraviti adekvatno kašnjenje između trenutka uključenja i isključenja LED.

Uobičajeni princip po kome se implementira kašnjenje jeste da mikrokontroleru zadate da odbrojava od prethodno postavljenog broja do nule, i da u trenutku dostizanja nule prekine odbrojavanje. Nulta vrednost, dakle, predstavlja kraj i izlazak iz petlje, nakon čega PIC nastavlja sa izvršavanjem daljeg programa.

Najpre treba definisati konstantu koja će predstavljati inicijalnu vrednost brojača. Nazovite je BROJAC. Zatim treba odlučiti kolika će biti stvarna vrednost ovog broja. Najveći broj koji se može zadati je 255 odnosno FFh.

Međutim, kao što je napomenuto u prethodnom poglavlju, naredba equ pridružuje imena običnim brojevima, ne praveći razliku između adresa registra i bitova. Zato ćete, ukoliko probate pridružiti vrednost FFh, dobiti grešku pri kompajliranju programa. Ovo se dešava jer je memorijska adresa FFh neimplementovana u PIC16F84, i zbog toga joj ne možete pristupiti. Pa kako onda pridružiti željeni broj? Za to je potrebno da sa strane sagledate situaciju. Ukoliko imenu BROJAC pridružite, na primer adresu 0Ch, ona će ukazivati na adresu registra opšte namene. Prilikom dobijanja napona napajanja, sve neiskorišćene memorijske lokacije u PIC, postavljaju se na vrednost FFh. Zato će BROJAC imati vrednost FFh.

Ali, kako onda podesiti BROJAC na drugu vrednost? Sve što trebate uraditi je da ubacite željenu vrednost u taj registar. Na primer, ukoliko želite da BROJAC sadrži vrednost 85h, ne možete napisati BROJAC equ 85h, jer će Vam onda BROJAC ukazivati na TRISA registar. Zato treba da uradite ovo:

```
movlw      85h      ; Prvo stavite broj 85h u W registar
movwf      0Ch      ; Onda ga prebacite u 0Ch registar
```

Sada možete napisati BROJAC equ 0Ch. BROJAC će biti jednak vrednosti 85h.

Korišćenje neinicijalizovanih (FFh) vrednosti u mikrokontroleru može dovesti do zabune prilikom kasnije analize programa. Verovatno ćete se pitati kako ste mogli smanjiti vrednost u registru bez njegove prethodne inicijalizacije. U primerima u ovom uputstvu one su korišćene zbog štednje memorijskih lokacija mikrokontrolera. Ukoliko ih Vi budete koristili u svojim programima obavezno to naglasite odgovarajućim komentarima.

Znači, prvo trebate definisati konstantu:

```
BROJAC      equ    0Ch
```

Dalje trebate smanjivati vrednost BROJAC sve dok ne dostigne vrednost 00f. Unutar PIC postoji instrukcija kojom se može ovo uraditi, uz malu pomoć instrukcije goto i labele. Ta instrukcija ima sledeći oblik:

```
decfsz      BROJAC, 1
```

Instrukcija DECFSZ f,d (eng. **D**ecrement **f**, **S**kip if **z**ero) znači “Smanji vrednost registra f (u ovom slučaju BROJAC odnosno 0Ch) za 1 i stavi vrednost u W registar ako je d=0, odnosno u registar f ako je d=1. Ukoliko je rezultat nakon smanjenja jednak 0 preskoči sledeću instrukciju”.

Kako se odredište rezultata koristi u dosta instrukcija, a kao što ste već naučili lakše je pamtit i imena umesto brojeva, moguće je i bitu odredišta dati ime, na sledeći način:

```
W      equ 00000000b
F      equ 00000001b
```

Mnogo reči, za jednu instrukciju. Pogledajte njenu primenu u praksi.

```
BROJAC      equ    0Ch
Pet          decfsz  BROJAC, F
              goto    Pet
              Nastavljamo odavde
```

Ovde je najpre postavljena konstanta BROJAC na FFh. U sledećem redu definisana je labela nazvana Pet, i zadana je decfsz instrukcija. Decfsz BROJAC,F smanjuje vrednost BROJAC za 1, i rezultat vraća u BROJAC. Ona takođe proverava da li je vrednost BROJAC jednaka 0. Ukoliko nije, program se nastavlja od sledeće linije. U sledećoj liniji imamo instrukciju goto, koja vraća izvršenje programa na decfsz instrukciju.

To se ponavlja sve dok vrednost BROJAC ne bude jednaka nuli. Onda program preskače sledeću instrukciju (u ovom primeru goto) i nastavlja od mesta gde je napisano “Nastavljamo odavde”. Kao što vidite, ovim se program “vrti” u jednom mestu tačno određeno vreme (kao brojanje u žmurkama), pre nego što nastavi dalje. To se naziva “prazna petlja”. Ukoliko je potrebna veća pauza, možete ubaciti jednu petlju unutar druge. Što više petlji, veća pauza. Biće vam potrebne bar dve, da biste mogli videti da LED treperi.

U prošlom poglavlju to nije naglašeno, ali svakom programu potrebno je definisati početak (adresu unutar PIC16F84 sa koje počinje snimanje koda) i kraj (posle koga nema više instrukcija). Početak programa se definiše sledećom pseudo instrukcijom

```
ORG 00h
```

Sada ubacite ove petlje u program, i dovršite ga praveći pravi program sa komentarima:

```

; ***** Imenovanje registra *****
STATUS      equ  03h  ; Adresa STATUS registra
TRISA       equ  85h  ; Adresa TRISA registra
PORTA       equ  05h  ; Adresa PORTA registra
BROJAC1     equ  0Ch  ; Prvi brojač za praznu petlju. Inicijalno FFh
BROJAC2     equ  0Dh  ; Drugi brojač za praznu petlju. Inicijalno FFh
RP0         equ  00100000b ; Oznaka bita 5 STATUS registra
F          equ  00000001b ; Oznaka odredišta rezultata instrukcije

; ***** Podešavanje porta *****
      org      00h
      bsf      STATUS, RP0      ; Prebacuje nas u BANK1
      movlw    00h              ; Postavlja sve pinove PORTA
      movwf    TRISA            ; kao izlazne
      bcf      STATUS, RP0      ; Vraća nas u BANK0

; ***** Uključi LED *****

Poc    movlw    05h              ; Uključi LED stavljajući vrednost 00100b
      movwf    PORTA            ; u W registar, a zatim iz njega u PORTA

; ***** Prva petlja aktivna pri uključenju LED *****

Pet1   decfsz   BROJAC1, F ; Smanji BROJAC1 za 1
      goto     Pet1      ; Ukoliko je BROJAC1 jednak 0, nastavi dalje
      decfsz   BROJAC2, F ; Smanji BROJAC2 za 1
      goto     Pet1      ; Ukoliko je BROJAC2 jednak 0, nastavi dalje,
                        ; u suprotnom vrati se na pocetak naše petlje.
                        ; Ovaj brojač broji nadole od 255 do 0, 255 puta
                        ; To je tzv. petlja unutar petlje.

; ***** Kašnjenje završeno. Sada isključi LED *****

      movlw    00h              ; Ugasi LED stavljajući 0 u W registar, a zatim
      movwf    PORTA            ; u PORTA

; ***** Druga petlja aktivna pri isključenju LED *****

Pet2   decfsz   BROJAC1, F ; Druga petlja drži LED ugašenom
      goto     Pet2      ; dovoljno dugo da bi se to mogli videti
      decfsz   BROJAC2, F ;
      goto     Pet2      ;

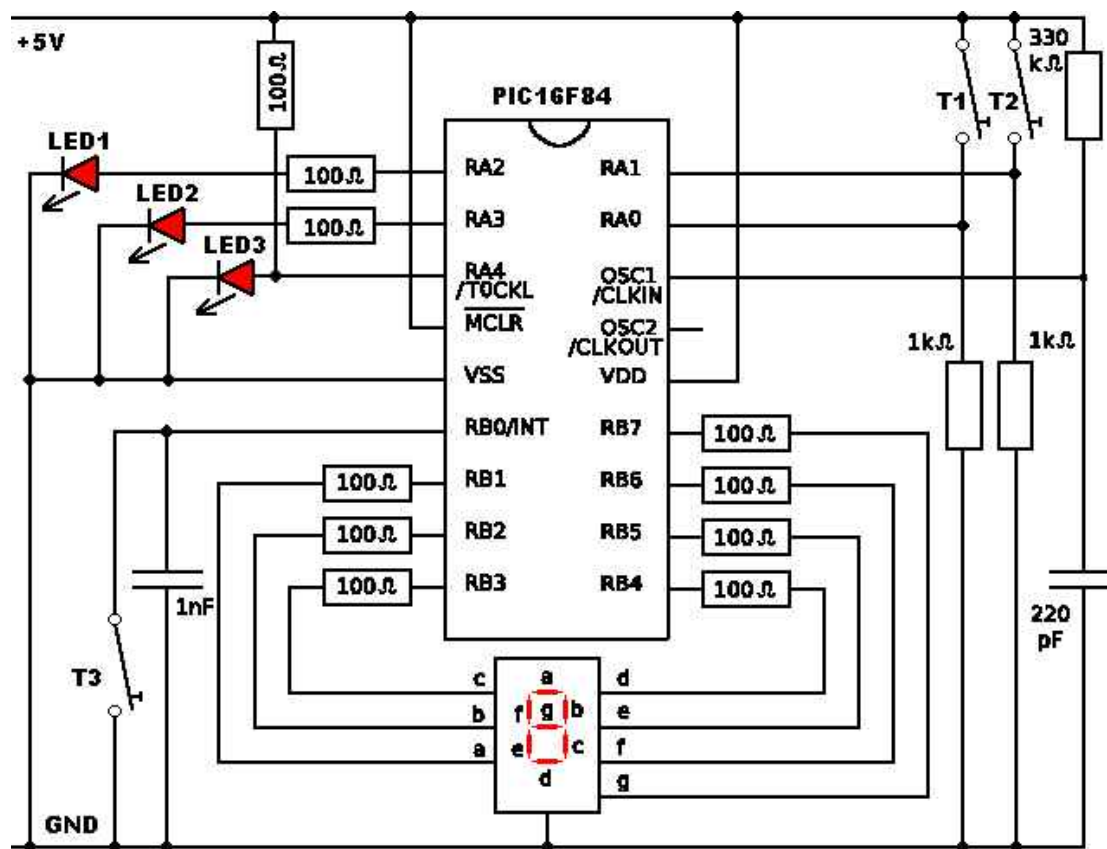
; ***** Sada se program vraća na početak *****

      goto     Poc          ; Vraćanje na početak programa i tamo
                        ; ponovo uključivanje LED

; ***** Kraj programa *****
      end                  ; Kraj programa.

```

Naravno, želećete da isprobate program da bi videli da li zaista radi. Evo električne šeme koju trebate napraviti za test svih programa u ovom uputstvu. Ovu šemu možete smatrati za mini razvojni sistem mikrokontrolera PIC16F84.



Za ovaj primer dovoljni su Vam LED1, njen otpornik od 100Ω, i RC oscilatorno kolo koje čine otpornik od 330kΩ i kondenzator od 220pF.

Čestitamo. Upravo ste napravili svoj prvi program, i napravili kolo kojim se LED uključuje i isključuje. Za sada ste sledeći ova uputstva naučili 7 od ukupno 35 instrukcija, i već kontrolišete ulazno izlazni port. Takođe ste naučili upotrebu

U sledećem poglavlju naučiti kako da svoj kod kompajlirate i snimate u PIC.

5. MPLAB Assembler

U ovom poglavlju upoznaćete se sa MPLAB razvojnim okruženjem, i specijalnim funkcijama njegovog editora.

Sa sajta <http://www.microchip.com/> trebate skinuti programski paket MPLAB. On ima integrisan editor (umesto Windows Notepada), assembler, simulator i drajvere za svoje programatore. Na žalost nas sa dial-up vezom, sam paket teži oko 30Mb.

Funkcija editora bi trebalo da vam je jasna. Generisanje čistog .asm fajla. Assembler služi da tekstualne instrukcije iz .asm fajla pretvori u mašinski kod koji PIC razume. U protivnom, umesto instrukcije `movlw 01h` trebali bi pisati binarni mašinski kod `11000000000001`.

Verovatno vam je već dosadilo da iznova i iznova pišete `STATUS equ 03h` i ostala standardna imena. MPLAB assembler ima mogućnost učitavanja fajla koji sadrži sva ova imena. Još bolje, u sebi ima i predefinisane obrasce (standard code template) za upis assemblerskog programa sa zaglavljem, uobičajenim pseudo instrukcijama, posebnim delom za interapte (više o njima kasnije)...

MPLAB ima jedan poznat bag koji se ogleda u nemogućnosti rada sa putanjom do foldera dužom od 64 karaktera. Stoga direktno u C: napravite folder „Moji programi“. U njega kopirajte sledeći fajl:

`P16f84.inc` - sadrži sve standardne labele

Ovaj fajl trebalo bi da se nalazi u folderu

`C:\Program Files\Microchip\MPASM Suite\P16f84.inc`

Onda u Vašem tekst editoru napravite sledeće zaglavlje,

```
.....
; Autor
; Datum
; Verzija
; Naslov
;
; Opis
;
;
;
;
.....

list      p=16F84      ; Definiše upotrebljeni mikrokontroler

#include  <p16F84.inc>  ; Ubacuje nazive registra u program
```

```

    __CONFIG    _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                ; Podešava konfiguracione bitove.
                ; Više o njima mnogo kasnije.

    ORG    00h                                ; Definiše start programa

; Prostor za vaše programe.

    end                                        ; Kraj programa

```

i snimite ga u folder „C:\Moji programi“ sa nazivom „Zagl.asm”.

Unutar foldera „C:\Moji programi“ napravite folder „Proba”.

Ovo zaglavlje možete koristiti za svaki novi program koji pravite, međutim uvek ga možete promeniti, tako da odgovara Vašim specifičnim potrebama.

Ukoliko zavirite u strukturu `p16F84.inc` fajla (F3 iz Total Commandera), možete primeriti da su u njemu imenovani svi registri, pa čak i bitovi pojedinih registra. Ako vam njegova trenutna struktura iz bilo kog razloga ne odgovara (npr. u ranijim verzijama assemblera je `OPTION_REG` bio samo `OPTION`), i ovaj fajl možete editovati po sopstvenom nahođenju.

Sada uključite MPLAB IDE. Ne obazirite se na prozore koje je otvorio, već sledite sledeću proceduru: Project, Project Wizard, Next, izaberite PIC16F84, Next

Izaberite (ukoliko već nije) Microchip MPASM Toolsuite.

Kliknite na „MPASM Assembler (Mpasmwin.exe)“, i proverite da li se nalazi u folderu C:\Program Files\Microchip\MPASM Suite. Ukoliko ga nema, locirajte ga ručno klikom na Browse. Ostala dva fajla Vam nisu bitna.

Next, Upišite ime projekta (npr. Proba) i izaberite putanju do ranije napravljenog foldera C:\Moji programi\Proba

Next, Uđite u C:\Moji programi. Kliknite naizmenicno na `P16f84.inc`, `Zagl.asm`, i na Add, tako da se oba fajla pojave u desnom prozoru. Nakon toga u desnom prozoru čekirajte kockice sa njihove leve strane. Njihovim čekiranjem se ti fajlovi kopiraju u trenutni projekat (u folderu Proba), tako da originali ostaju nepromenjeni za idući put.

Next i kliknite na Finish.

U novootvorenom prozoru „Proba.mcw“ duplo kliknite na `Zagl.asm`. Otvara se prozor editora u kome možete pisati Vaš program.

Možda vam ovaj postupak trenutno izgleda komplikovano, ali u praksi je potrebno ponoviti ga 2 do 3 puta da bi postao to što i jeste. Obična rutina.

Sa svim ovim podešavanjima kompletan program bi trebao izgledati otprilike ovako:

```

;~~~~~
; Autor      Pera Perić
; Datum      26.4.2007
; Verzija    5.9
; Naslov     Treptanje LED
;
; Opis       Ovim programom omogućeno je
;            treptanje LED1 na RA2 pinu
;
;~~~~~

; ***** Inicijalizacija assemblera *****

list      p=16F84      ; Definiše upotrebljeni mikrokontroler
#include   <p16F84.inc> ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
;         ; Podešava konfiguracione bitove.

; ***** Imenovanje registra *****

BROJAC1    equ    0Ch ; Prvi brojač za praznu petlju. Inicijalno FFh
BROJAC2    equ    0Dh ; Drugi brojač za praznu petlju. Inicijalno FFh

; ***** Podešavanje porta *****

org        00h
bsf       STATUS, RP0 ; Prebacuje nas u BANK1
movlw     00h         ; Postavlja sve pinove PORTA
movwf     TRISA       ; kao izlazne
bcf       STATUS, RP0 ; Vraća nas u BANK0

; ***** Uključi LED *****

Poc movlw   05h         ; Uključi LED stavljajući vrednost 00100b
    movwf   PORTA       ; u W registar, a zatim iz njega u PORTA

; ***** Prve petlja aktivna pri uključenju LED *****

Pet1 decfsz  BROJAC1, F ; Smanji BROJAC1
    goto     Pet1       ; Ukoliko je BROJAC1 jednak 0, nastavi dalje
    decfsz  BROJAC2, F ; Smanji BROJAC2
    goto     Pet1       ; Ukoliko je BROJAC2 jednak 0, nastavi dalje,
                        ; u suprotnom vrati se na pocetak naše petlje.
                        ; Ovaj brojač broji nadole od 255 do 0, 255 puta
                        ; To je tzv. petlja unutar petlje.

```

```

; ***** Kašnjenje završeno. Sada isključi LED *****

    movlw    00h          ; Ugasi LED stavljajući 0 u W registar, a zatim
    movwf    PORTA        ; u PORTA

; ***** Druga petlja aktivna pri isključenju LED *****

Pet2 decfsz    BROJAC1, F ; Druga petlja drži LED ugašenom
     goto     Pet2       ; dovoljno dugo da bi se to mogli videti
     decfsz    BROJAC2, F ;
     goto     Pet2       ;

; ***** Sada se program vraća na početak *****

     goto     Poc        ; Vraćanje na početak programa i tamo
                           ; ponovo uključivanje LED
; ***** Kraj programa *****

end                               ; Kraj programa.

```

Kao što ste i sami pretpostavili pseudo instrukcija `#include <p16F84.inc>` ubacuje fajl `p16F84.inc` ispred Vašeg assemblerskog programa. Sam assembler će pri pretvaranju instrukcija u mašinski kod uzeti iz fajla nazive samo onih imenovanih registra koji se koriste u Vašem assemblerskom programu. Na taj način program je pregledniji, Vama je skraćeno vreme pisanja programa (za imenovanje registra), a program nije ništa duži nego inače.

Sada možete assemblerirati program. Za asembliranje pritisnite taster F10 na tastaturi. U novootvorenom prozoru „Output“ videćete kratki opis uspešnosti operacije. Možete slobodno ignorisati Message [302]. Ona vas podseća da trebate biti u BANK1 pri podešavanju TRISA registra. Da se ona ne bi pojavljivala, trebalo bi dati jedan naziv STATUS registru kada je u BANK0, a drugi kada je u BANK1. Ukoliko Vas ova poruka nervira, možete u delu sa inicijalizacijom assemblera ubaciti "ERRORLEVEL -302".

U „Output“ prozoru najbitnija je zadnja poruka. Ona mora biti „BUILD SUCCEEDED“.

U folderu „Proba“ sada se nalazi par fajlova. Bitni su vam jedino oni sa ekstenzijom .err, .lst i .hex.

U fajlu sa .err ekstenzijom, možete videti spisak grešaka i upozorenja nastalih pri asembliranju programa.

Fajl sa .lst ekstenzijom predstavlja detaljan pregled svih upotrebljenih instrukcija, komentara... Iz njega možete videti u kom trenutku je tačno nastupila greška u asembliranju.

Fajl sa .hex ekstenzijom predstavlja fajl spreman za snimanje u PIC. Postupak snimanja, naučićete u narednim poglavljima.

5. Simulator

Simulacija je proces simuliranja izvršavanja instrukcija iako još ništa konkretno nije hardverski napravljeno. MPLAB IDE ima u sebi integrisan simulator.

Da biste uključili simulator idite na Debugger, Select Tool, MPLAB SIM. U toolbaru će se pojaviti nekoliko novih ikonica. Kliknite na treću (dve strelice nadesno) i pratite izvršavanje programa u prozoru sa .asm fajlom. Vidite kako je došao do petlje i kako neprestano izvršava dve iste instrukcije. Na drugoj ikoni (Halt) pauzirate program. Na prvoj (jedna strelica nadesno) se program izvršava maksimalnom brzinom, ali bez animiranosti njegovog izvršavanja. Četvrtom ikonom se kada je program u pauziranom stanju vrši prelaz na sledeću instrukciju, korak po korak. Zadnjom ikonom se vrši reset programa, i njegovo izvršavanje počinje iz početka. Registri pri tome uglavnom zadržavaju svoje vrednosti.

Da biste videli aktuelni sadržaj registra BROJAC1 i BROJAC2 možete ići na View, File Registers. U prozoru koji se otvorio možete videti da se registar na adresi 0Ch (koju je uzeta za BROJAC1) u koracima smanjuje do 0. Kada dostigne 0, smanjuje se BROJAC2 na adresi 0Dh.

Možda vam je brzina izvršavanja programa previše spora? Povećajte je na Debugger, Settings, Animation / Realtime Updates pod Animate step time. Umesto 500mS stavite npr 50. Možete slobodno staviti i manje vrednosti. Donju granicu određuje brzina vašeg kompjutera.

Ukoliko Vas interesuje realna brzina kojom bi program radio u stvarnim uslovima, možete je proveriti preko Debugger, Stopwatch opcije. Dovoljno je izabrati brzinu oscilatora u Debugger, Settings, Osc / trace opciji. Za date vrednosti RC oscilatora (330kΩ i 220pF), ona je oko 300KHz.

Iako je prikaz stanja svih bitova PORTA registra veoma lako implementirati, ukoliko je potrebno simulirati program koji treba reagovati na spoljne signale (tipa pritisnut prekidač T1) stanje se iz osnova menja. Da bi to bilo moguće mora se definisati tačno vreme u koje će se na određeni pin dovesti logička jedinica. Uopšte ne postoji mogućnost klika na odgovarajući bit registra, kako bi mu se promenilo stanje na ulazima mikrokontrolera.

Ukoliko Vam je takva pogodnost neophodna, možete skinuti PIC Simulator IDE sa <http://www.oshonsoft.com/>. Nažalost, može se koristiti samo 30 dana pre isteka probnog perioda. Budući da ga je napravio čovek sa naših prostora, možda pri kupovini dobijete popust.

Još jedan odličan simulator je Proteus sa <http://www.labcenter.co.uk/>. Ni on nije besplatan.

Simuliranje programa pre pravljenja gotove pločice može Vam uštedeti dosta kasnijih problema. Jeftinije je virtuelno isprobati program.

6. Programatori

Za narezivanje .hex fajla u PIC neophodan vam je programator kao hardverski deo i određeni softver preko koga se vrši programiranje.

Nemojte se zaleteti pa odmah kupiti najskuplji Microchipov programator. Funkcija programatora je programiranje PIC mikrokontrolera možda 5-6 puta dnevno. Da li će on biti na USB ili paralelnom portu uopšte nije bitno. Jedino bih preporučio izbegavanje programatora sa serijskim portom, zbog nejednakosti njegovih karakteristika na raznim matičnim pločama. Ukoliko imate iskustva u pravljenju elektronskih kola, možete napraviti jedan od programatora sa sajta <http://www.icprog.com/>. Na istom sajtu nalazi se i pripadajući softver za programiranje. Proguglajte malo. Možda naiđete i na nešto lakše.

Ukoliko pak želite kupiti programator, možete pogledati sledeće sajtove:

- Allpic programator
<http://www.geocities.com/sinelyu/prodaja/programatori.html>
Okolo 10 €
- EU-PRPIC-01
<http://www.interhit.co.yu/>
oko 3.700 din.
- Razvojni sistemi – programator i razvojni sistem u jednom
<http://www.mikroelektronika.co.yu/domestic/>
Od 100 do 140 €

I opet pre kupovine pretražite internet. Vaš novac je u pitanju.

Unutar mikrokontrolera nalaze se takozvani konfiguracioni bitovi. Njima se određuju hardverske specifičnosti mikrokontrolera. Iako su inicijalno ubačeni u __CONFIG pseudoinstrukciju, neki programatori je ignorišu i postupaju po sopstvenim setovanjima.

Za PIC16F84 konfiguracioni bitovi su:

Watchdog Timer – dozvoljava odnosno zabranjuje upotrebu WDT u programu
Power Up Timer Enable – prouzrokuje kašnjenje pri dovođenju napona napajanja da bi se oscilator mikrokontrolera stabilizovao
Code Protection – onemogućava iščitavanje koda iz PIC – kopiranje programa
Oscillator – omogućuje izbor jednog od 4 standardna tipa oscilatora, odnosno
RC – (eng. **R**esistor **C**ondensator) 1 otpornik i 1 kondenzator
LP – (eng. **L**ow **P**ower) Za kristalne oscilatore male potrošnje 32KHz-200KHz
XT – Za klasične kristalne oscilatore 100KHz-4MHz
HS – (eng. **H**igh **S**peed) Za kristalne oscilatore veće brzine 4MHz-10MHz

Upotrebu Watchdog Timera naučićete kasnije. Za skoro sve primere u ovom uputstvu, on treba biti isključen – WDT OFF.

Power Up Timer Enable bit je za RC oscilator upotrebljen u kolu preporučljivo ostaviti uključenim – PWRTE ON.

Code Protection je poželjno isključiti, kako biste mogli verifikovati program nakon programiranja. Međutim, ukoliko trebate prodati PIC sa vašim programom na koji ste utrošili 3 meseca rada, obavezno ga uključite. U ovom uputstvu, neka bude isključen - CP OFF

PIC16F84 može koristiti više vrsta oscilatora. Najeftiniji ali i najnestabilniji u pogledu tačnosti frekvencije je svakako RC oscilator, poput onog na šemi. Preporučene vrednosti kondenzatora su od 20 pF do 300nF, a otpornika od 5 do 100kΩ. Sam mikrokontroler je inače prilično tolerantan prema ovim vrednostima, pa je moguć njegov rad i bez ikakvog kondenzatora (koristeći parazitnu kapacitivnost štampanih vodova i pinova). Mikrokontroler je potrebno podesiti za izbor RC oscilatora u ovom uputstvu preko - OSC RC.

Ukoliko nije potrebna velika brzina izvršavanja instrukcija, a bitna je potrošnja i stabilnost frekvencije, (npr. sat na baterije) PIC treba raditi sa 32KHz-200KHz kristalom. Na manjem taktu PIC troši manje struje nego inače. Za ovakvu vrstu oscilatora treba podesiti konfiguracioni bit oscilatora na – OSC LP.

Ukoliko je bitna brzina, a i cena PIC16f84, dobar izbor bi bio kristalni oscilator od 4MHz. Za njega je potrebno konfiguracioni bit podesiti kao - OSC XT.

A ukoliko je brzina imperativ, mogu se kupiti verzije PIC16F84 mikrokontrolera koje mogu bez problema raditi na taktu većem od 4MHz. Nažalost, one su i skuplje. Njihov konfiguracioni bit potrebno je podesiti kao – OSC HS.

Dosta je toga o assembleru, simulatoru i hardveru. Vreme je za učenje novih instrukcija. U sledećem poglavlju videćete kako možete koristiti podprograme koji će Vam pomoći da program bude manji i jednostavniji.

7. Podprogrami

Podprogram je deo koda, ili program, koji možemo pozvati kao takav kad god je potreban. Podprogrami se koriste u slučajevima višestrukog izvršavanja jedne iste funkcije, kao na primer rutinu sa praznom petljom za izazivanje kašnjenja. Prednosti korišćenja podprograma su u lakoći menjanja vrednosti brojača jednom unutar podprograma, nego npr. deset puta kroz glavni program, i u smanjenju količine memorije koju program zauzima unutar PIC.

Pogledajte strukturu jednog podprograma:

```
BROJAC      equ 0Ch      ; Uzima vrednost FFh

Pet          decfsz      BROJAC, F      ; Smanjuje i proverava
             goto       Pet           ; sve do 0
             return      ; Povratak iz podprograma
```

Prvo trebate dati podprogramu ime, i u ovom slučaju izabrano je da to bude “Pet”. Onda se upisuje kod koji se izvršava u okviru podprograma. U ovom slučaju, to je kašnjenje za program sa LED. Na kraju se podprogram završava RETURN instrukcijom. Da biste pozvali podprogram iz glavnog programa, jednostavno upišite CALL instrukciju i labelu početka podprograma.

Kada glavni program dođe do dela sa instrukcijom CALL k (eng. **Call** Subroutine) “pozovi podprogram”, gde je k adresa ili labela podprograma, on skače na mesto na kojem se nalazi podprogram. Tamo nastavlja sa izvršavanjem komandi unutar podprograma sve do nailaska na instrukciju RETURN (eng. **Return** from Subroutine) “povratak iz podprograma”. Po nailasku na RETURN program nastavlja izvršavanje od instrukcije koja se nalazi iza CALL k instrukcije. Možete pozvati podprogram koliko god puta želite, i zbog toga se korišćenjem podprograma smanjuje ukupna dužina programa. Ipak postoje dve stvari na koje morate obratiti pažnju. Prvo, u glavnom programu, sve konstante moraju biti definisane pre nego što se upotrebe. One se mogu definisati u samom podprogramu, ili na početku glavnog programa. Najpraktičnije je definisati ih na početku programa. Na taj način ćete znati da su sve na istom mestu. Drugo, morate biti sigurni da će izvršavanje glavnog programa zaobići podprogram. Ukoliko stavite podprogram na kraj vašeg programa i zaboravite da stavite instrukciju goto koja bi nastavila izvršavanje dalje od podprograma, program će nastaviti sa izvršavanjem i izvršiti i podprogram želeli Vi to ili ne. PIC ne pravi razliku između podprograma i glavnog programa.

Prepravite program za treperenje LED tako da sada koristite podprogram sa praznom petljom. Videćete da izgleda jednostavnije.

```
; ***** Inicijalizacija i imenovanje *****
list          p=16F84          ; Definiše upotrebljeni mikrokontroler
#include       <p16F84.inc>      ; Ubacuje nazive registra u program
__CONFIG      _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                                     ; Podešava konfiguracione bitove.

BROJAC1       equ 0Ch          ; Prvi brojač za petlju. Inicijalno FFh
BROJAC2       equ 0Dh          ; Drugi brojač za petlju. Inicijalno FFh
```

```

; ***** Podešavanje porta *****
    org      00h
    bsf      STATUS, RP0      ; Prebacuje nas u BANK1
    movlw    00h              ; Postavlja pinove PORTA
    movwf    TRISA            ; kao izlazne
    bcf      STATUS, RP0      ; Vraća nas u BANK0
; ***** Uključi LED *****
Poc movlw    04h              ; Uključi LED stavljajući vrednost 00100b
    movwf    PORTA            ; u W registar, a zatim iz njega u PORTA
; ***** Dodaj kašnjenje *****
    call     Pet
; ***** Kašnjenje završeno, sada isključi LED *****
    movlw    00h              ; Isključi LED stavljajući vrednost 00000b
    movwf    PORTA            ; u W registar, a zatim iz njega u PORTA
; ***** Dodaj još jedno kašnjenje *****
    call     Pet
; ***** Sada se vrati na početak našeg programa *****
    goto     Poc
; ***** Podprogram za kašnjenje *****
Pet  decfsz   BROJAC1, F      ; Ove dve petlje služe za brojanje  nadole od
    goto     Pet              ; 255 do 0, 255 puta, omogućavajući nam da
    decfsz   BROJAC2, F      ; možemo videti kako LED treperi
    goto     Pet
    return                                ; Povratak iz podprograma
; ***** Kraj programa *****
    end                                ; Kraj programa.

```

Možete videti da je korišćenjem podprograma smanjena veličina programa. Svaki put kada je potrebno napraviti pauzu, bez obzira na to da li LED svetli ili ne, jednostavno se pozove podprogram. Na kraju podprograma program se vraća na red iza CALL instrukcije. U gornjem primeru, prvo se uključuje LED. Onda se poziva podprogram. Nakon povratka iz podprograma može se isključiti LED. Opet se poziva podprogram, i po povratku iz njega, izvršava se sledeća instrukcija, odnosno “goto Start”.

Bitno je napomenuti i to da je moguće pozivanje drugog potprograma unutar prvog. Na primer ovo je potpuno moguće:

```

Pocetak  call  Prog1          ; Pozivanje prvog podprograma
          goto  Pocetak        ; Vraćanje na početak
Prog1     call  Prog2          ; Pozivanje drugog podprograma
          return               ; Povratak iz prvog podprograma
Prog2     return               ; Povratak iz drugog podprograma

```

Ovako se može ići samo do osmog nivoa dubine na šta se mora obratiti pažnja. Za one koje interesuje, originalan program bio je dug 120 bajtova. Korišćenjem podprograma, smanjen je na 103 bajta. Ovo možda i ne izgleda mnogo, ali imajte u vidu da je u PIC moguće koristiti samo 1024 bajta. Svaki bajt je važan.

U sledećem uputstvu videćete kako se očitavaju podaci sa porta.

8. Čitanje iz ulazno izlaznih portova

Do sada ste samo pisali po PORTA registru da biste mogli uključivati i isključivati LED. Sada ćete videti kako se može čitati sa ulazno/izlaznih pinova PORTA registra. Na taj način možete na pinove mikrokontrolera povezati spoljno elektronsko kolo, i programirati PIC da različito reaguje u zavisnosti od njegovih signala.

Ako se priselite prethodnih poglavlja, da bi podesili ulazno izlazni port, trebate se prebaciti iz BANK0 u BANK1. Najpre to uradite:

```
bsf STATUS, RP0
```

Sada, da biste podesili pin na PORTA registru kao izlazni, treba da pošaljete 0 u njegov TRISA registar. Da biste ga podesili kao ulazni, šaljete 1.

```
movlw    01h          ; Stavi 00001b u W, a zatim u
movwf    TRISA        ; TRISA registar. Tako je bit 0 ulazni.
bcf      STATUS, RP0
```

Ovim je bit 0 PORTA registra podešen kao ulazni. Sada je potrebno da proverite da li je pin 17 na integrisanom kolu PIC16F84 na logičkoj jedinici ili nuli. Za ovo koristimo jednu od instrukcija BTFSC i BTFSS.

BTFSC f,b (eng. **Bit Test f, Skip if Clear**) instrukcija znači “Izvrši testiranje bita b na registru f. Ako je taj bit jednak 0, preskoči sledeću instrukciju“. BTFSS f,b (eng. **Bit Test f, Skip if Set**) znači “Izvrši testiranje bita b na registru f. Ako je taj bit jednak 1, preskoči sledeću instrukciju“

Koju instrukciju ćete koristiti zavisi od toga kako želite da program reaguje kada pročitate podatak sa ulaza. Na primer, ako jednostavno čekate da ulaz postane 1, možete iskoristiti instrukciju BTFSS ovako:

Početak programa

```
.
.
.
Test      btfss    PORTA, 0
          goto     Test
```

Nastavi odavde

```
.
.
```

Program će nastaviti izvršavanje od “Nastavi odavde” samo ukoliko je bit 0 u PORTA setovan, odnosno ukoliko je na pinu 17 prisutan visoki logički nivo.

Napišite sada program kojim će LED treptati jednom brzinom, a ukoliko se pritisne prekidač, ona će treptati duplo sporije. Možete probati da sami napišete ceo program, da biste videli da li ste shvatili postupak. Koristi se ista elektronska šema kao i ranije, sa dodatim prekidačem T1 i njegovim otpornikom od 1kΩ.

```

; ***** Inicijalizacija i imenovanje *****

    list      p=16F84          ; Definiše upotrebljeni mikrokontroler
    #include  <p16F84.inc>      ; Ubacuje nazive registra u program
    __CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                                ; Podešava konfiguracione bitove.
    BROJAC1   equ  0Ch          ; Prvi brojač petlje. Inicijalno FFh
    BROJAC2   equ  0Dh          ; Drugi brojač petlje. Inicijalno FFh
    org       00h

; ***** Podešavanje porta *****

    bsf       STATUS, RP0      ; Prebacuje nas u BANK1
    movlw     01h              ; Stavi 00001b u TRISA. Tako je bit 0
    movwf     TRISA            ; ulazni, a bit 2 izlazni.
    bcf       STATUS, RP0      ; Vraća nas u BANK0

; ***** Uključenje LED *****

Poc  movlw     04h              ; Uključi LED stavljajući 00100b u
    movwf     PORTA           ; W registar, a zatim iz njega u PORTA

; ***** Proveri da li je prekidač zatvoren *****

    btfsc     PORTA, 0         ; Uzmi vrednost iz bita 0 PORTA. Ako je 0
                                ; preskoči prvu pauzu. Ako je 1 prodi kroz
    call      Pet              ; obe pauze, kao da se ništa ne događa

; ***** Dodavanje još jedne pauze *****

    call      Pet              ; Ova pauza se uvek izvršava

; ***** Pauza završena. Sada isključi LED *****

    movlw     00h              ; Isključi LED stavljajući vrednost 00000b
    movwf     PORTA           ; u W registar, a zatim iz njega u PORTA

; ***** Proveri da li je prekidač i dalje zatvoren *****

    btfsc     PORTA, 0         ; Uzmi vrednost iz bita 0 PORTA. Ako je 0
                                ; preskoči prvu pauzu. Ako je 1 prodi kroz
    call      Pet              ; obe pauze, kao da se ništa ne događa

; ***** Dodavanje još jedne pauze *****

    call      Pet              ; Ova pauza se uvek izvršava

; ***** Pauza završena. Sada se vrati na početak *****

    goto      Poc              ; Vrati se na početak i ponovo upali LED

```

```

; ***** Ovde počinje podprogram *****

Pet  decfsz    BROJAC1, F ; Ove dve petlje služe za brojanje  nadole od
    goto      Pet        ; 255 do 0, 255 puta, omogućavajući nam da
    decfsz    BROJAC2, F ; možemo videti kako LED treperi
    goto      Pet
    return                                ; Povratak iz podprograma

; ***** Kraj programa *****

                                end                ; Kraj programa.

```

Program najpre uključuje LED. Onda proverava da li je prekidač zatvoren. Ukoliko jeste (na ulazu je 1), program se izvršava kao da se ništa nije desilo, i prolazi kroz oba poziva za podprogram. Ukoliko nije (na ulazu je 0), prvi poziv za podprogram se preskače, pa je pauza u tom slučaju ista kao kod ranije napisanih programa. Isto se dešava i kada se isključi LED.

Možete kompajlirati i isprobati ovaj program. Ipak da vas odmah upozorim. Kolo i program neće impresionirati nekog koga ne interesuje programiranje mikrokontrolera. Zato se nemojte uznemiravati kada svojoj porodici i prijateljima pokažete kako možete menjati brzinu treptanja LED prekidačem, a oni za to pokazuju veoma malo interesovanja.

Ukoliko ste iz početka pratili ova uputstva, onda biste možda voleli da znate da ste do sada naučili 10 od ukupno 35 instrukcija za PIC16F84! Sve su naučene samo uključivanjem i isključivanjem LED.

9. Efikasno korišćenje memorije

Do sada ste programirali PIC da uključuje i isključuje LED. Onda ste upravljali njegovim radom dodavši mu prekidač, i menjajući brzinu treptanja diode. Jedini problem je u tome što je program bio dugačak, i bespotrebno je trošio memoriju. To je bilo u redu, u toku objašnjavanja instrukcija, ali mora da postoji bolji način da se ovo reši. Pa i postoji (znali ste da ovo sledi, zar ne?).

Hajde da vidimo kako ste uključivali i isključivali LED.

```
movlw      04h
movwf      PORTA
movlw      00h
movwf      PORTA
```

Prvo ste ubacili 04h u naš W registar, zatim prosledili na PORTA da uključite LED. Da bi je isključili ubacili ste 00h u W, i onda ga prebacili na PORTA registar. Između ovih rutina pozivan je podprogram za kašnjenje da biste mogli videti da LED treperi. Znači dvaputa su prebacivana 2 seta podataka (jednom u W registar, a onda u PORTA) i dva puta je pozivan podprogram (jednom za uključenje i jednom za isključenje).

Kako se ovo može efektnije implementirati? Koristeći logičku operaciju zvanu XOR.

XOR (eng. Exclusive **Or**) operacija izvršava logičku funkciju “ekskluzivno ili” na registru koji odredite, sa vrednošću koju mu date. Trebalo bi ovo malo bolje objasniti.

Ukoliko imate dva ulaza i jedan izlaz, izlaz će biti 1 samo ukoliko su mu ulazi različiti. Ukoliko su isti (obe 0 ili obe 1), izlaz će biti 0. Ovde je tablica istinitosti za one koji više vole takav opis.

A	B	F	A	00101110
0	0	0	B	10010110
0	1	1	F	10111000
1	0	1		
1	1	0		

XOR operacija može se naći u instrukcijama PIC16F84 u dva oblika.

1. XORLW k gde vršimo XOR operaciju nad registrom W i konstantnom vrednošću k. Vrednost konstante k mora biti u opsegu od 0 do 255 (00h – FFh). Rezultat operacije će biti u W registru.
2. XORWF f,d gde XOR operaciju izvršavamo nad vrednošću W registra i vrednošću registra čija je adresa označena sa f. Sa „d“ je označeno gde će biti smešten rezultat XOR operacije. Ukoliko je d=0, rezultat će biti u W registru, a ukoliko je d=1, rezultat će biti smešten u f registru.

Pogledajte šta bi se dogodilo ukoliko B preuzima vrednost svog prethodnog rezultata, u slučaju da se menja samo vrednost A:

A	B	F
0	0	0
0	0	0
1	0	1
1	1	0
1	0	1

Ukoliko zadržite vrednost A na 1, i izvršite ekskluzivno ili sa prethodnim izlazom kao B, izlaz će se promeniti. Za one koji ne mogu videti to iz tablice istinitosti, evo prikaza sa binarnim vrednostima:

	0	Trenutni izlaz
ekskluzivno ili sa 1	1	Novi izlaz
ekskluzivno ili sa 1	0	Novi izlaz

Nadam se da možete videti da se kada se primeni ekskluzivno ili nad 1 i prethodnim izlazom, u stvari menja vrednost izlaza sa 0 na 1 i sa 1 na 0.

Dakle, sada su Vam za uključenje i isključenje LED potrebne samo dve linije:

```
movlw    04h
xorwf    PORTA, F
```

Ovde je vrednost 04h (00100b) ubačena u W registar. Onda se izvršava ekskluzivno ili nad ovim brojem, i nad vrednošću koja se nalazi u PORTA. Ukoliko je bit 2 bio 1, postaće 0, a ukoliko je bio 0 postaće 1.

Pogledajte primer ove operacije da biste videli kako radi u binarnom brojnom sistemu.

```
PORTA
00100b
xorwf    00000b
xorwf    00100b
xorwf    00000b
xorwf    00100b
```

Čak ne morate ni učitavati vrednost 00100b vrednost u W registar svaki put. Možete to uraditi jednom na početku, i dalje se samo vraćati na xorwf instrukciju. Takođe nama potrebe ni za podešavanjem početne vrednost u PORTA registru. Zbog čega? Pa, ukoliko je po uključanju napajanja vrednost njegovog bita 1 bila 0, promenićemo je. Sa druge strane ukoliko je bila 1, opet ćemo je promeniti.

Pogledajte sada novi assemblerski kod. Prvi je originalni program za treperenje LED, a u drugom je dodat prekidač.

Treptuća LED

; ***** Inicijalizacija i imenovanje *****

```
list      p=16F84          ; Definiše upotrebljeni mikrokontroler
#include   <p16F84.inc>     ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
          ; Podešava konfiguracione bitove.
BROJAC1    equ  0Ch        ; Prvi brojač petlje
BROJAC2    equ  0Dh        ; Drugi brojač petlje
org        00h
```

; ***** Podešavanje porta *****

```
bsf        STATUS, RP0     ; Prebacuje nas u BANK1
movlw      00h             ; Postavlja pinove PORTA
movwf      TRISA           ; kao izlazne.
bcf        STATUS, RP0     ; Vraća nas u BANK0
movlw      04h             ; Ubaci 00100b u W registar
```

; ***** Uključenje i isključenje LED *****

```
Poc  xorwf   PORTA, F      ; Promeni stanje LED
```

; ***** Pauza *****

```
call      Pet
```

; ***** Pauza završena. Vрати se na početak programa *****

```
goto      Poc             ; Vрати se na početak
```

; ***** Podprogram za kašnjenje *****

```
Pet  decfsz  BROJAC1, F    ; Ove dve petlje služe za brojanje nadole od
goto  Pet      ; 255 do 0, 255 puta, omogućavajući nam da
decfsz  BROJAC2, F    ; možemo videti kako LED treperi
goto  Pet
return                                     ; Povratak iz podprograma
```

; ***** Kraj programa *****

```
end                                     ; Kraj programa
```

Treptuća LED sa prekidačem

; ***** Inicijalizacija i imenovanje *****

```
list      p=16F84      ; Definiše upotrebljeni mikrokontroler
#include  <p16F84.inc>  ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
          ; Podešava konfiguracione bitove.
BROJAC1   equ  0Ch      ; Prvi brojač petlje
BROJAC2   equ  0Dh      ; Drugi brojač petlje
org       00h
```

; ***** Podešavanje porta *****

```
bsf       STATUS, RP0   ; Prebacuje nas u BANK1
movlw     01h           ; Stavi 00001b u TRISA. Tako je bit 0
movwf     TRISA         ; ulazni, a bit 2 izlazni.
bcf       STATUS, RP0   ; Vraća nas u BANK0
movlw     04h           ; Ubaci 00100b u W registar
```

; ***** Uključenje i isključenje LED *****

```
Poc  xorwf     PORTA, 1   ; Promeni stanje LED
```

; ***** Proveri da li je prekidač zatvoren *****

```
btfscc    PORTA, 0      ; Uzmi vrednost iz bita 0 PORTA. Ukoliko je 0
          ; preskoči prvu pauzu. Ukoliko je 1 prodi kroz
call      Pet           ; obe pauze, kao da se ništa ne događa
```

; ***** Dodavanje još jedne pauze *****

```
call      Pet           ; Ova pauza se uvek izvršava
```

; ***** Pauza završena. Vрати se na početak programa *****

```
goto      Poc           ; Vрати se na početak
```

; ***** Podprogram za kašnjenje *****

```
Pet  decfsz    BROJAC1, F ; Ove dve petlje služe za brojanje nadole od
    goto      Pet        ; 255 do 0, 255 puta, omogućavajući nam da
    decfsz    BROJAC2, F ; možemo videti kako LED treperi
    goto      Pet
    return                      ; Povratak iz podprograma
```

; ***** Kraj programa *****

```
end                      ; Kraj programa
```

Nadam se da ste uočili da je upotrebom samo jedne instrukcije prilično smanjena veličina programa. U stvari, da biste lakše uočili ovu razliku, prikazni su nazivi ova dva programa, izmene koje su izvršene, i veličine programa u tablici ispod:

Program	Izmena	Veličina (bajtova)
Treptuća LED	Original	120
Treptuća LED	Dodat podprogram	103
Treptuća LED	Dodata ekskluzivno ili funkcija	91
LED sa prekidačem	Original	132
LED sa prekidačem	Dodata ekskluzivno ili funkcija	124

Vidite da ste ne samo naučili nove instrukcije, već ste i smanjili veličinu programa.

10. Logičke operacije

Jednu logičku operaciju (XOR) ste već naučili. Sada ćete se upoznati sa još tri.

AND „logičko I“ operacija daće na svom izlazu logičku 1 samo ukoliko su joj na oba ulaza dovedene logičke 1. Ovde je njena tablica istinitosti.

A	B	F	A	01101011
0	0	0	B	10100101
0	1	0	F	00100001
1	0	0		
1	1	1		

Kao i XOR i AND operacija se može naći u dva oblika.

1. ANDLW k koja izvršava AND operaciju nad registrom W i konstantnom vrednošću k. Rezultat operacije će biti u W registru.
2. ANDWF f,d koja izvršava AND operaciju nad vrednošću W registra i vrednošću f registra. Sa „d“ je označeno gde će biti smešten rezultat operacije. Ukoliko je d=0, rezultat će biti u W registru, a ukoliko je d=1, rezultat će biti smešten u f registru.

Rezultat IOR (inkluzivno ili) operacije biće 1 ako je bar 1 od bitova (ili oba) u stanju logičke 1.

A	B	F	A	01101011
0	0	0	B	10100101
0	1	1	F	11101111
1	0	1		
1	1	1		

I ona se javlja u dva oblika.

1. IORLW k koja izvršava IOR operaciju nad registrom W i konstantnom vrednošću k. Rezultat operacije će biti u W registru.
2. IORWF f,d koja izvršava IOR operaciju nad vrednošću W registra i f registra. Ukoliko je d=0, rezultat će biti u W registru, a ukoliko je d=1, rezultat će biti u f registru.

Zadnja logička operacija koju podržava PIC16F84 je komplement. Komplementacija ili invertovanje predstavlja logičku operaciju u kojoj svaki bit u bajtu menja svoju vrednost.

A	F	A	01101111
0	1	F	10010000
1	0		

Za razliku od ostalih logičkih operacija komplementiranje se obavlja nad samo jednim bajtom. Zbog toga i ova operacija ima samo jedan oblik instrukcije

COMF f,d vrši komplementiranje svih bitova u registru f. Ukoliko je d=0 rezultat će biti u W, a ukoliko je 1 u f registru.

Verovatno uočavate da ste ovu logičku operaciju mogli upotrebiti u ranijem programu umesto XOR. Jedina razlika je u tome što bi se onda invertovali SVI bitovi, a ne samo onaj koji je potreban.

Sve do sada naučene logičke operacije menjaju vrednost bit 2 STATUS registra u slučaju da je rezultat njihove operacije jednak nuli. Kako je bit 2 po tome specifičan on ima svoju posebnu oznaku „nulti bit“ (eng. **Zero flag**). Termin „flag“ koristi se za označavanje bitova na koje nemamo direktnog uticaja već ih mikrokontroler sam setuje ili resetuje u zavisnosti od rezultata zadnje operacije.

Dakle ukoliko je rezultat XOR operacije sledeći:

A	00011010b
B	00011010b
F	00000000b

Zero flag STATUS registra će biti setovan. Ovo možete iskoristiti za brzo poređenje ulaza tako što u W registar postavite željeni oblik bitova a onda izvršite XOR operaciju nad njim i željenim portom. Ukoliko je ulazni signal na portu jednak vrednosti postavljenoj u W registru, Zero flag će biti setovan.

Iako nema mnogo smisla Zero flag možete setovati i operacijom komplementovanja.

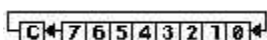
A	11111111
F	00000000

11. Rotacija

RLF (**R**otate **L**eft **f** Through **C**arry) instrukcija rotira ulevo bitove unutar zadatog registra. Sintaksa instrukcije je sledeća:

RLF *f,d* pri čemu je *f* registar nad čijim sadržajem se obavlja rotacija. Ukoliko je *d*=0, rezultat operacije će biti u *W* registru, a ukoliko je *d*=1, rezultat će biti u registru *f*.

Šta radi ova instrukcija? Ukoliko npr. u registru 08h imate vrednost 00000001b nakon instrukcije RLF 08h,1 u registru 08h će se naći vrednost 00000010b. Nakon još jedne iste instrukcije u registru će biti vrednost 00000100b. Kao što vidite ova instrukcija rotira sadržaj registra ulevo. A šta kada 1 dođe do kraja – pitate se vi. Neće se izgubiti. Nakon rotacije ulevo osmi bit smešta se u tzv. “bit prekoračenja” (eng. Carry flag). To je bit 0 STATUS registra.



Dakle kada jedinica u primeru dođe do kraja, događa se sledeće:

	Carry flag	Bitovi 08h registra
		76543210
rlf 08h, F	0	01000000
rlf 08h, F	0	10000000
rlf 08h, F	1	00000000
rlf 08h, F	0	00000001
rlf 08h, F	0	00000010

Vidite da je Carry flag na neki način ovde iskorišćen kao deveti bit registra.

Ukoliko vam je to potrebno možete ubaciti 0 ili 1 u Carry flag pre rotacije instrukcijama `bsf STATUS,C` i `bcf STATUS,C` (C je imenovani bit 0 STATUS registra). Tako će se nova vrednost pojaviti na bitu 0 nakon `rlf` instrukcije. Preporučljivo je ovo uraditi neposredno pre prve rotacije kako na bitu 0 nakon rotacije ne bi dobili nepoznatu vrednost. Ne utiče samo RLF instrukcija na Carry flag.

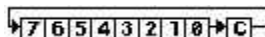
Interesantno je da se rotacijom ulevo obavlja i množenje brojeva sa 2.

C	76543210		
0	01011001b	=	89
0	10110010b	=	178
1	01100100b	=	356 (računajući i Carry flag)

Ukoliko vam je potrebno množenje sa 2 većih brojeva možete koristiti uzastopnu rotaciju dva ili više registra. Carry flag iz prvog registra preneće se na početak drugog.

Kada već postoji instrukcija za rotaciju ulevo, logično bi bilo da postoji i instrukcija za rotaciju bitova udesno. Postoji!

RRF (Rotate Right f Through Carry)



Sintaksa instrukcije je ista kao i kod RLF.

Kako se rlf instrukcijom vrši množenje sa 2, rrf instrukcijom vrši se deljenje sa 2, a pred prelazak bita u Carry flag događa se sledeće:

	Carry flag	Bitovi 08h registra
		76543210
rrf 08h, F	0	00000010
rrf 08h, F	0	00000001
rrf 08h, F	1	00000000
rrf 08h, F	0	10000000
rrf 08h, F	0	01000000

Instrukcije RLF i RRF mogu se upotrebiti za simulaciju trčućeg svetla. U tom slučaju oblik instrukcije bi bio:

```
rlf  PORTB, F    ; za rotaciju ulevo i
rrf  PORTB, F    ; za rotaciju udesno.
```

U instrukcijama je upotrebljen je PORTB jer on ima 8 ulazno izlaznih pinova dok ih PORTA ima samo 5.

Da biste isprobali upravo naučene instrukcije postavite prekidač između RA1 i +5V. Sada isprogramirajte PIC sledećim programom:

```
; ***** Inicijalizacija i imenovanje *****

list      p=16F84          ; Definiše upotrebljeni mikrokontroler
#include   <p16F84.inc>     ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                        ; Podešava konfiguracione bitove.
DATA      equ 0Ch          ; Registar za privremene podatke
org       00h

; ***** Podešavanje porta *****

bsf      STATUS, RP0      ; Prebacuje nas u BANK1
movlw    03h              ; Stavi 00011b u TRISA. Tako su bit 0 i
movwf    TRISA            ; bit 1 ulazni, a bit 2 izlazni.
bcf      STATUS, RP05     ; Vraća nas u BANK0

; ***** Testiranje prekidača *****

Poc rlf    PORTA, W        ; Rotiraj sadržaj PORTA i stavi rezultat u W
                        ; registar, Tako je bit 0 došao na mesto bita 1
```

```

xorwf      PORTA, W      ; Izvrši ekskluzivno ili nad PORTA i W
                        ; registrom, i stavi rezultat u W (interesuje nas
                        ; ekskluzivno ili nad bitom 1 ova dva registra).

; ***** Prebacivanje rezultata na izlaz *****

movwf      DATA        ; Premesti sadržaj iz W u DATA jer se rotacija
                        ; ne može izvršiti direktno nad W registrom
rlf        DATA, W      ; Rotiraj sadržaj. Tako je bit 1 došao na mesto
                        ; bita 2 (na mesto izlaznog bita) i vrati rezultat
                        ; u W registar.
movwf      PORTA        ; Stavi rezultat u PORTA. Kako su bit 0 i bit 1
                        ; ulazni, neće reagovati, a na bitu 2 će se
                        ; pojaviti rezultat ekskluzivno ili operacije nad
                        ; ulaznim stanjem.
goto       Poc          ; Povratak na početak.

; ***** Kraj *****

end                ; Kraj programa.

```

Kompajlirajte program, programirajte PIC i isprobajte njegov rad. Iz komentara se može lako zaključiti šta program radi.

Evo šta će se dogoditi u programu u slučaju da su oba prekidača zatvorena: PIC uzima vrednost iz PORTA. Kako su oba tastera pritisnuta ona je 00011b. Ta vrednost se rotira, i postaje 00110b. Rezultat se smešta u W registar. Dalje se izvršava ekskluzivno ili nad tom vrednošću i trenutnom vrednošću PORTA koji je i dalje 00011b.

Ako se prisetite tablice istinitosti ekskluzivne ili funkcije, lako možete zaključiti da je rezultat sledeći:

Tablica istinitosti			bitovi	43210
A	B	F	W registar	00110b
0	0	0	PORTA	00011b
0	1	1	rezultat	00101b
1	0	1		
1	1	0	posle rotacije	01010b

Sada se rezultat 00101b prebacuje u privremeni registar 0Ch gde se rotira. Rotirana vrednost je 01010b, i ona se preko W registra smešta u PORTA. Kako su na PORTA bit 0 i bit 1 ulazni, vrednost koja će doći na njih biće ignorisana, a bit 3 (LED2) nas ionako ne interesuje i njegova vrednost se može ignorisati. Kao što vidite na kraju svega bit 2 sadrži vrednost ekskluzivno ili funkcije (u ovom slučaju 0). Kako je ovaj bit izlazni, LED1 neće svetleti. Možete proveriti rezultat funkcije pritiskanjem prekidača po tablici istinitosti. Program je napisan tako da inicijalna vrednost Carry flaga nema uticaja na njegovo izvršavanje, pa njegovo brisanje ovde nije potrebno.

Kako je ovaj program dat kao ilustracija korišćenja instrukcija nema potrebe za ograničenjem setovanja bita 3 PORTA registra (LED2). U praksi taj pin ne bi bio povezan.

Prepravite sada program po sledećem:

Umesto	xorwf	PORTA, W	; Izvrši ekskluzivno ili nad PORTA i W
stavite	andwf	PORTA, W	; Izvrši i operaciju nad PORTA i W

Program radi isto što i ranije, s tom razlikom što umesto XOR primenjuje AND operaciju. Pritiskanjem prekidača možete videti da će LED1 svetleti jedino ukoliko su oba prekidača (T1 i T2) zatvorena (na oba ulaza dovedena logička jedinica).

Ukoliko u programu zamenite instrukciju ANDWF instrukcijom IORWF možete isprobati rad ove operacije. LED1 će svetleti ukoliko je uključen bar jedan prekidač.

Do sada ste naučili 19 od 35 instrukcija. Čestitamo. To je više od polovine.

12. Aritmetičke operacije

Već ste videli kako PIC16F84 može množiti i deliti sa dva u binarnom brojnem sistemu. Sada ćete naučiti kako može brojati unapred i unazad kao i sabirati i oduzimati.

INCF (eng. **I**ncrement **f**) instrukcija služi za inkrementaciju. Njen oblik je:

INCF f,d pri čemu d određuje gde će ići rezultat (u W ili F)..

Ovom instrukcijom se vrši povećanje registra f za 1. Ukoliko je npr. vrednost registra 08h bila 00h, nakon instrukcije inc 08h,F njegova vrednost će postati 01h, a nakon sledeće iste instrukcije 02h.

A šta će biti kada se dođe do FFh? - pitate se vi. Odmah ćete videti.

	Bitovi	STATUS
	76543210	Zero flag
	11111110	0
incf 08h, F	11111111	0
incf 08h, F	00000000	1
incf 08h, F	00000001	0

Ukoliko vam je potrebna inkrementacija do većih vrednosti možete detektovati stanje Zero flaga, i onda pokrenuti sledeći registar za inkrementaciju. Ili za još veće brojeve možete kao u primeru sa praznom petljom staviti petlju unutar petlje.

Instrukcija suprotna ovoj je DECF (eng. **D**ecrement **f**). Ova instrukcija smanjuje vrednost registra f za 1, odnosno dekrementuje ga. Sintaksa joj je ista kao kod INCF instrukcije. U slučaju da brojač dođe do 0 desiće se sledeće:

	Bitovi	STATUS
	76543210	Zero flag
	00000001	0
decf 08h, F	00000000	1
decf 08h, F	11111111	0
decf 08h, F	11111110	0

Dva specijalna oblika ovih instrukcija su: DECFSZ f,d koju ste već naučili u praznoj petlji i INCFSZ f,d (**I**ncrement **f**, **S**kip if **z**ero) koja povećava sadržaj registra f za 1 po istom principu kao INCF instrukcija, i koja preskače sledeću instrukciju u slučaju da je rezultat operacije jednak 0.

```
Poc incfsz 08h, F ; Povećaj vrednost u registru 08h za 1,
goto Poc ; i vrati se na početak. Ukoliko se nakon povećanja
; dobije 0, preskoči goto instrukciju i nastavi dalje.
```

Ove instrukcije za razliku od običnih INCF i DECF ne utiču na Zero flag.

ADD “saber” operacija sabira dva broja.

Pogledajte kako se izvršava operacija sabiranja u binarnom brojnem sistemu:

A	B	F			76543210
0	0	0	A	01010100	84
0	1	1	B	01110110	118
1	0	1	F	11001010	202
1	1	10			

Sigurno ste primetili da se ovde javlja prenos jedinice u bit veće težine u slučaju prekoračenja. To može dovesti do problema u slučaju da je zbir dva broja veći od 255.

A 11001101b 205 ; Ovde je rezultat operacije sabiranja
 B 10111000b 184 ; veći od maksimalne vrednosti
 F 1 10000101b 389 ; koja može biti u jednom registru (255)

Kod ovakvih slučajeva bit prekoračenja smešta se u Carry flag (bit 0 STATUS registra).

U instrukcijama za PIC operacija sabiranja ima dva oblika:

ADDLW k sabira sadržaj W registra i konstantne vrednosti k. Rezultat će se naći u W registru.

ADDWF f,d sabira sadržaj W registra i registra f. Sa d se određuje gde će se naći rezultat.

SUB (eng. **Sub**stract) „oduzmi“

Da biste naučili oduzimanje u binarnom brojnem sistemu moraćete prethodno naučiti komplement dvojke. To će vam biti utoliko lakše ukoliko se priselite običnog komplementovanja (ili invertovanja ako vam je tako lakše).

A	F	A	01101111
0	1	F	10010000
1	0		

Komplement dvojke je običan komplement kome je dodat broj 1, odnosno $F = \text{COMF}(A) + 1$.

A 01101111
 10010000 + 1
 F 10010001

Oduzimanje B od A se unutar mikrokontrolera obavlja sabiranjem broja A sa komplementom dvojke broja B.

```

      A      11001101b      ; 205
      B      10111000b      ; 184
COMF (B) 01000111b
COMF (B)+1 01001000b
      F 1 00010101b      ; 21

```

Pogledajte sada primer oduzimanja većeg od manjeg broja: 56-179

A	00111000b				; 56
		B	10110011b		; 179
		COMF (B)	01001100b		
COMF (B) +1	01001101b				
F	10000101b				; -123

Testiranjem Carry flaga možete utvrditi da li je rezultat operacije oduzimanja pozitivan ili negativan (tačnije negativan ili 0), a testiranjem Zero flaga da li je rezultat 0.

SUBLW k oduzima sadržaj W registra od konstantne vrednosti k. Rezultat će se naći u W registru.

SUBWF f,d oduzima sabira sadržaj W registra od registra f. Sa d se određuje gde će se naći rezultat.

Aritmetičke operacije sabiranja i oduzimanja pored Carry i Zero flaga utiču još i na tzv. DC flag (eng. **D**igit **C**arry), odnosno bit 1 STATUS registra. On će biti setovan pri prekoračenju donjeg četvorobitnog dela bajta (tzv. nibla). Menja se po istom principu kao i Carry flag.

	C	DC			C	DC		
	A		1111	1111	A		0000	1111
ADD	B		0000	0001	ADD	B	0000	0001
	F	1 1	0000	0000		F	0 1	0001 0000

Deljenje bajta na niblove ima smisla ukoliko je potrebno jednim niblom prikazivati jednu cifru, a drugim drugu (npr u digitalnom časovniku za minute gornji nibl od 0 do 5, a donji od 0 do 9). Tako je moguće odjednom promeniti obe cifre slanjem jednog podatka na PORTB. Uvećanje samo gornjeg nibla rešava se sabiranjem sa 00010000b.

13. Malo o hardveru

Do sada ste pisali i pisali programe. Vreme je da se malo pozabavite unutrašnjom strukturom mikrokontrolera. Nakon toga neke stvari će vam biti jasnije.

Sve instrukcije sa kojima radi mikrokontroler su četrnaestobitne. Međutim, unutar samih instrukcija nalazi se kod instrukcije (po kome se npr. instrukcija CALL razlikuje od MOVLW) i operand, koji predstavlja podatak sa kojim radimo (bit, bajt, adresa registra). Postoje i instrukcije koje nemaju operand (npr. RETURN).

Pre izvršenja svake instrukcije PIC gleda sadržaj registra PCL (02h) i PCLATH (0Ah). Oni (tačnije bitovi 0-5 PCLATH + 0-8 PCL) čine jedinstven trinaestobitni PC (Program Counter) registar. U njemu je smeštena adresa izvršavanja instrukcije. Po izvršavanju tekuće instrukcije, PC se uvećava za 1, i prelazi se na sledeću instrukciju. Ukoliko je tekuća instrukcija instrukcija uslovnog skoka (DECFSZ, INCFSZ, BTFSC i BTFSS instrukcije), rezultat instrukcije (tačno ili netačno – 1 ili 0) dodaje se na PC, čime je omogućen skok preko sledeće instrukcije. Međutim ukoliko je tekuća instrukcija GOTO, ona će u PC postaviti novi adresu, i izvršenje programa nastaviće se odatle. Specijalan slučaj predstavlja korišćenje CALL instrukcije. PIC mora na neki način upamtiti mesto sa koga je skočio na podprogram. Ta informacija čuva se u steku.

Stek je interni deo mikrokontrolera (kao uostalom i W registar) kome mogu pristupiti jedino instrukcije skoka i povratka iz podprograma ili interapt rutine (više o njoj kasnije). Njega možete zamisliti kao cev zatvorenu sa jedne strane u koju redom ubacujete klikere. Crveni, zeleni, žuti i plavi. Očigledno, ne možete izvući zeleni kliker ukoliko prethodno ne izvučete plavi pa žuti.

Pri nailasku na CALL instrukciju, adresa naredne instrukcije se stavlja na stek, i vrši se izmena PC. Mikrokontroler skače na podprogram, i tamo nastavlja izvršenje programa sve do instrukcije povratka. Po nailasku na instrukciju povratka iz podprograma, mikrokontroler preuzima adresu povratka sa steka, i smešta je u PC. Dalje izvršenje programa nastavlja se odatle. Međutim, stek nije beskonačan. U njega se može upisati maksimalno 8 adresa za povratak (7 ukoliko se koriste interapti). Nakon toga nove adrese povratka prebrisaće početne (crveni pa zeleni kliker...), što će prouzrokovati nepravilan rad programa. Ovo ograničenje ne bi trebalo preterano da Vas brine. U praksi uglavnom nema potrebe za podprogramima koji idu više od druge, ili treće dubine.

Iako PIC nema set instrukcija kojima bi se mogle koristiti tabele u programu, moguće je implementirati ih upotrebom PCL i PCLATH registra i instrukcije RETLW k.

Retlw k (**R**eturn with **L**iteral in **W**) instrukcija omogućava povratak iz podprograma (isto kao i RETURN instrukcija), pri čemu se nakon povratka bajt k nalazi u W registru. Za obične podprograme to i nije neka velika pogodnost, ali je za tabele neophodno. Tabele se koriste pozicionirajući PCL registar na odovarajuću instrukciju.

U primeru koji sledi tabela se koristi za odabir odgovarajućeg šablona za crtanje brojeva od 0 do 9 na sedmosegmentnom LED displeju.

```

; ***** Inicijalizacija i imenovanje *****

        list          p=16F84          ; Definiše upotrebljeni mikrokontroler
        #include      <p16F84.inc>      ; Ubacuje nazive registra u program
        __CONFIG      _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                                           ; Podešava konfiguracione bitove.
BROJ     equ          0Ch              ; Promenljiva za broj koji se prikazuje (00h-09h)
        org           00h

; ***** Podešavanje porta *****

        bsf           STATUS, RP0      ; Prebacuje u BANK1
        movlw         01h              ; Postavlja pinove PORTB i to izlazni
        movwf         TRISB           ; za pinove 7 do 13, a ulazni za pin 6
        bcf           OPTION_REG, NOT_RBP ; Uključeni interni „pull up“
                                           ; otpornici na PORTB
        bcf           STATUS, RP0      ; Vraća u BANK0

; ***** Inicijalizacija *****

Ini      clrfs         BROJ            ; Postavljanje na 0
        call          Disp            ; Prikaz na displeju

; ***** Testiranje prekidača *****

Cek1     btfs         PORTB, 0         ; Testiranje otpuštenosti prekidača
        goto         Cek1            ; Nije otpušten
Cek2     btfs         PORTB, 0         ; Još jedno testiranje zbog imunosti na
        goto         Cek2            ; eventualna varničenja kontakta.
Cek3     btfs         PORTB, 0         ; Testiranje pritisnutosti prekidača
        goto         Cek3            ; Nije pritisnut
Cek4     btfs         PORTB, 0         ; Još jedno testiranje zbog imunosti na
        goto         Cek4            ; eventualna varničenja kontakta.

; ***** Brojanje *****

        movlw         F6h             ; F7h u W (F6h + 09h + 1 = setovan Carry flag)
        addwf         BROJ, W         ; Saberi sa BROJ, rezultat u W
        btfs         STATUS, C        ; Da li je došlo do prekoračenja (Carry flag)?
        goto         Ini              ; Jeste, resetuj brojač
        incf         BROJ, F          ; Nije, inkrementuj brojač,
        call          Disp            ; prikaži rezultat i.
        goto         Cek1            ; nastavi sa testiranjem i brojanjem

```



```

; ***** Podprogram za prikaz na LED displeju *****

Disp  call      Tabl      ; Obrazac za prikaz u W
      movwf     PORTB     ; Prikaži na displeju
      return      ; Povratak iz podprograma

; ***** Tabela *****

Tabl  movf      BROJ, 0    ; U promenljivoj BROJ nalazi se vrednost od
      addwf     PCL, 1     ; 00h do 09h. Ta vrednost se dodaje na PCL.
      retlw     01111110b ; Obrazac za crtanje cifre 0 - 0
      retlw     00001100b ; Obrazac za crtanje cifre 1 - 1
      retlw     10110110b ; Obrazac za crtanje cifre 2 - 2
      retlw     10011110b ; Obrazac za crtanje cifre 3 - 3
      retlw     11001100b ; Obrazac za crtanje cifre 4 - 4
      retlw     11011010b ; Obrazac za crtanje cifre 5 - 5
      retlw     11111010b ; Obrazac za crtanje cifre 6 - 6
      retlw     00001110b ; Obrazac za crtanje cifre 7 - 7
      retlw     11111110b ; Obrazac za crtanje cifre 8 - 8
      retlw     11011110b ; Obrazac za crtanje cifre 9 - 9

; ***** Kraj programa *****

      end          ; Kraj programa.

```

U inicijalizacionom delu ste verovatno uočili upotrebu nove instrukcije. Instrukcija `clrf,f` (eng. **C**lear **f**ile) koristi se brisanje željenog registra (popunjavanjem sa 00h). Za razliku od seta instrukcija: `movlw 00h` i `movwf BROJ`, `clrf BROJ` resetuje Zero flag.

Još jedna instrukcija koju ste zapazili na početku tabele je `movf f,d` (eng. **M**ove **f**). Ona vrednost iz registra `f` prebacuje u `W` (pri `d=0`) ili u taj isti registar `f` (pri `d=1`). Zbog čega bi iko želeo da sadržaj registra upiše u taj isti registar? U praksi se ta mogućnost koristi jedino pri testiranju registra na vrednost 00h, jer `movf` instrukcija utiče na Zero flag.

U programu ste mogli zapaziti i korišćenje `OPTION_REG` registra. Resetovanje njegovog bita 7 (**P**ort **B** **P**ull **U**p) omogućava povezivanje internih otpornika mikrokontrolera na svaki od ulaza porta B prema naponu napajanja mikrokontrolera. Time je eliminisana potreba za priključenjem eksternih otpornika na ulaze mikrokontrolera. Interni otpornici spajaju se na svu ulazne pinove porta B. Pored te, `OPTION_REG` registar ima i razne druge funkcije, pa je stoga najpraktičnije ne menjati sadržaj njegovih ostalih bitova.

Duplo testiranje pritisnutosti prekidača je procedura koja se standardno primenjuje kod kompjuterskih tastatura. Na taj način sprečavaju se impulsi varničenja nastali od mehaničkih kontakata prekidača (eng. *debouncing*). Iz istog razloga paralelno sa prekidačem je povezan kondenzator. U ovom primeru on bi se mogao i izbaciti uz malu modifikaciju programa (veća pauza između testiranja stanja prekidača). Testiranje na otpuštenost, a zatim na pritisnutost prekidača sprečava nastavak brojanja ukoliko se prekidač neprekidno drži pritisnut.

Prikaz dekadnog broja realizovan je direktnim dovođenjem napona na pojedine segmente LED displeja. Ovi segmenti su na LED displeju označeni su slovima a, b, c, d, e, f i g. Segment dp (eng. **d**ecimal **p**oint) „decimalna tačka“ ovde nije iskorišćen. Segmentima displeja dovode se naponi sa pinova 7 do 13, a prekidač T3 je povezan između pina 6 i mase. Program je napravljen za rad sa displejem sa zajedničkom katodom. Međutim, može se prepraviti za rad sa displejem sa zajedničkom anodom prepravkom tabele na taj način što se svi bitovi u tabeli (nulti bit nema uticaja na rad programa) invertuju:

```
retlw      01111110b ; 0 promeniti u 10000001b
retlw      00001100b ; 1 promeniti u 11110011b
retlw      10110110b ; 2 promeniti u 01001001b
```

U tom slučaju potrebno je prepraviti električnu šemu tako da se zajednička anoda poveže na +5V.

U podprogramu za prikaz možete videti klasičan primer pozivanja drugog podprograma iz prvog.

U radu sa tabelama morate obratiti pažnju na dve stvari.

1. Morate biti sigurni da je broj koji sabirate sa PCL u tačno određenim granicama. U slučaju prekoračenja izvršavanje programa će se nastaviti sa slučajne memorijske adrese što će prouzrokovati nepravilan rad ili blokadu programa.
2. Kako je PC trinaestobitni registar čiji donji deo (0-255) je u PCL, a gornji (>255) u bitovima 0-5 PCLATH, potrebno je osigurati se da tabela ne izađe iz memorijskog bloka od 256 bajtova. Kod programa navedenog kao primer, to nije problem, jer je on ionako dug samo 33 bajta. Ukoliko je neophodno koristiti tabele sa većom količinom podataka, rešenje bi moglo biti u postavljanju više tabela. Veličinu bloka najjednostavnije možete proveriti brojanjem upotrebljenih instrukcija (izuzimajući pseudo instrukcije), ili analizom .lst fajla generisanog prilikom assembliranja.

Umesto tabele moguće je napraviti skok na proračunate memorijske lokacije `addwf PCL,F` instrukcijom, pri čemu se adresa na koju se treba skočiti čuva u W registru u obliku broja instrukcija koje se trebaju preskočiti da bi se došlo do željene. Ograničenja vezana za tabele važe i u ovom slučaju.

14. Watchdog tajmer

Pretpostavimo da ste napisali program koji se neprekidno izvršava na mikrokontroleru. Normalno, želeli bi da se osigurate da će program uvek nastaviti sa izvršavanjem bez obzira na to šta se sa njim događa. Uzmite u obzir sledeći slučaj. Pretpostavimo da PIC nadgleda određeni ulazni pin. Kada se na ovaj pin dovede logička 1, program skače na naredni deo i čeka da sledeći pin dobije logičku 1. Ukoliko se to ne desi, PIC će se vrteti u petlji i čekati. Izaći će iz petlje tek kada se na pinu pojavi 1. Pogledajte sada malo drugačiji primer. Pretpostavimo da ste napisali program, kompajlirali ga, čak i simulirali njegov rad u simulatoru. Međutim posle dugo vremena program se zaglavi u nekoj petlji (koliko ste samo puta Vi pritisnuli Ctrl Alt Delete kombinaciju na Windowsu). U oba slučaja potrebna Vam je neka vrsta reseta ukoliko se program zaglavi. To je svrha Watchdog (eng. pas čuvar) tajmera.

Watchdog tajmer (skraćeno WDT) nije ništa novo. Mnogi mikrokontroleri i mikroprocesori imaju ga u sebi. Ali kako on radi? Unutar mikrokontrolera nalazi se kapacitivno otporna (eng. **Resistor Condenzator**) mreža. RC mreža obezbeđuje jedinstven takt, nezavistan od takta mikrokontrolera. Kada se WDT uključi, njegov brojač počinje sa 00, uvećava se za 1 dok ne dostigne FFh. U trenutku prelaska sa FFh na 00h (što je FFh+1) PIC će se resetovati bez obzira na njegovo stanje. Jedini način na koji se može sprečiti ovaj reset, je periodično resetovanje WDT na 0 kroz Vaš program. Sada možete i sami uvideti da u slučaju zaglavljivanja mikrokontrolera u petlji WDT neće biti resetovan, što će prouzrokovati reset mikrokontrolera.

Da biste mogli uspešno koristiti WDT potrebno je da znate tri stvari. Prvo za koliko vremena će WDT preći sa 00 na FF, drugo kada ga trebate resetovati i treće kako podesiti softver programatora za njegovo uključjenje.

Vreme WDT

WDT ima vreme prelaska sa 00 na FF od oko 18mS. Ovo vreme zavisno je od nekoliko spoljnih faktora kao što su napon napajanja, temperatura mikrokontrolera i.t.d. Zbog lakšeg objašnjavanja zaokružiću ovo vreme na tačno 18mS. Ipak, ovo vreme može se produžiti.

<i>bit 2,1,0</i>	<i>Odnos</i>	<i>Vreme</i>
0, 0, 0	1:1	18mS
0, 0, 1	1:2	36mS
0, 1, 0	1:4	72mS
0, 1, 1	1:8	144mS
1, 0, 0	1:16	288mS
1, 0, 1	1:32	576mS
1, 0, 1	1:64	1,1S
1, 1, 1	1:128	2,3S

Unutar mikrokontrolera nalazi se tzv. postskalier (eng. posle podeliti). On se može programirati za deljenje WDT takta generisanog internom RC mrežom. Što se veći odnos deljenja koristi, takt WDT će biti sporiji.

Postskalier se nalazi u OPTION_REG registru (81h) na mestu od nultog do drugog bita. U tabeli je prikazan odnos bitova, odnos deljenja takta i vreme prekoračenja vrednosti WDT.

Upamtite da su ova vremena nezavisna od takta oscilatora mikrokontrolera. Mislite o ovim vremenima kao stvarnim (eng. real time), nasuprot taktu mikrokontrolera koji možete menjati fizičkom zamenom par delova u oscilatoru. Pretpostavimo da želite da WDT resetuje PIC nakon pola sekunde. Najbliža standardna vrednost je 576mS, odnosno 0,576S, pa je dovoljno samo poslati vrednost b101 u OPTION_REG registar.

```
movlw    b101 ; Ovo je 05h
movwf    81h  ; Adresa OPTION_REG registra.
```

Međutim nije baš sve tako jednostavno. Po inicijalizaciji PIC-a postskalera je pridružen običnom internom tajmeru (u njemu on funkcioniše kao preskaler „pre podeliti“. Više o tome kasnije). To znači da je potrebno prebaciti ga na WDT. Kako i interni i watchdog tajmer koriste iste bitove OPTION_REG registra Microchip preporučuje sledeću proceduru kako ne bi došlo do nehotičnog reseta PIC.

```
; ***** Inicijalizacija i imenovanje *****
list      p=16F84          ; Definiše upotrebljeni mikrokontroler
#include   <p16F84.inc>     ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
org       00h              ; Podešava konfiguracione bitove.

; ***** Podešavanje WDT *****
*         bsf          STATUS,RP0          ; BANK1
*         movlw        bxx0x0111 ; Izaberi tajmer i preskaler različit od 000
*         movwf        OPTION_REG
*         bcf          STATUS,RP0          ; BANK0
*         clrf         TMR0              ; Brisanje vrednosti internog tajmera i
                                         ; preskalera – pri upisu u tajmer,
                                         ; preskalera – pri upisu u tajmer,
                                         ; preskalera – pri upisu u tajmer,
bsf       STATUS,RP0          ; BANK1
movlw     bxxxx1111 ; Izaberi WDT, bez menjanja odnosa
                                         ; sada postskalera
movwf     OPTION_REG
clrwdt    ; Obrisati WDT
movlw     bxxxx1xxx ; Izaberi WDT, i postavi željeni odnos
                                         ; postskalera xxx. Za 0,576S to je bxxxx1101
movwf     OPTION_REG
bcf       STATUS,RP0          ; BANK0
```

U slučaju željenog odnosa različitog od 1:1 (vrednost postskalera 000) mogu se izbaciti instrukcije označene zvezdicom,

Vrednosti u OPTION_REG registru levo od preskalera označene sa x imaju specijalne funkcije koje ćete naučiti u delu sa običnim tajmerom i interaptima. Iz tog razloga im ovde nisu dodeljene konkretne vrednosti. Za sada je dovoljno da znate da se bitom 3 OPTION_REG registra vrši selekcija tajmera kome će biti pridružen delitelj po sledećem:

- 1 – postskalera WDT
- 0 – preskalera internog tajmera.

U primeru ste naučili upotrebu CLRWDT (eng. **C**lear **W**atchdog **t**imer) instrukcije. Pored resetovanja WDT, ona utiče i na određene bitove STATUS registra. Više o tome u poglavlju sa RESET funkcijom.

Vreme izvršavanja instrukcija

Kao što se verovatno sećate, PIC uzima eksterni takt sa oscilatora i deli ga sa 4. Ovo interno vreme naziva se instrukcijski ciklus. Ukoliko je kao izvor takta upotrebljen na primer kristal od 4MHz, PIC će izvršavati instrukcije brzinom od $4\text{MHz}/4=1\text{MHz}$, odnosno za jednu instrukciju koja traje 1 instrukcijski ciklus će mu trebati $1\mu\text{S}$, dok će mu za instrukciju koja traje 2 instrukcijska ciklusa trebati $2\mu\text{S}$. U prilogu ovog uputstva možete videti koliko instrukcijskih ciklusa zahteva koja instrukcija. Način da to zapamtite je prilično lak. Pretpostavite da sve instrukcije traju 1 instrukcijski ciklus. Ali, ukoliko instrukcija prouzrokuje nastavak programa sa neke druge adrese (promenjen je sadržaj PC), onda će ona trajati 2 instrukcijska ciklusa. Na primer MOVWF instrukcija traje 1 instrukcijski ciklus, jer samo premešta podatak sa jednog na drugo mesto. GOTO instrukcija traje 2 ciklusa jer prouzrokuje da PC skoči na neko drugo mesto u programu. RETURN instrukcija traje 2 ciklusa, jer se PC vraća na glavni program. Međutim, postoje 4 instrukcije koje mogu trajati 1 ili 2 instrukcijska ciklusa. To su DECFSZ, INCFSZ, BTFSC i BTFSS. One će preskočiti sledeću instrukciju u slučaju da je određen uslov ispunjen i onda će trajati 2 ciklusa. Ukoliko taj uslov nije ispunjen, izvršiće se naredna instrukcija kao da se ništa nije ni desilo, i onda će trajati 1 ciklus. Pogledajte sada sledeći program.

```
      movlw      02
      movwf      BROJAC
Pet    decfsz     BROJAC, F
      goto       Pet
      end
```

Prva instrukcija postavlja vrednost 2 u W. Ovo ne prouzrokuje menjanje PC, pa ona traje 1 instrukcijski ciklus. Sledeća instrukcija je slična. Ona takođe ne menja stanje PC, pa i ona traje 1 ciklus. Sada će sledeća instrukcija izvršiti prvo smanjenje registra BROJAC za 1 i rezultat „testa na vrednost 0“ = 0 pridružiti PC. Na ovaj način PC se ne menja, pa instrukcija traje 1 ciklus. Sledeća instrukcija je GOTO, i ona traje 2 ciklusa. Onda se ponovo vrši smanjenje registra BROJAC za 1, ali kako je sada rezultat „testa na vrednost 0“ = 1, PC će promeniti svoju vrednost na PC+1, što troši 2 instrukcijska ciklusa. Znači ovaj program zauzeće ukupno 7 instrukcijskih ciklusa. Uz kristal od 4MHz program će trajati

$4\text{MHz} / 4\text{takta} = 1\mu\text{S}$ po instrukcijskom ciklusu, što za 7 ciklusa iznosi $7 * 1\mu\text{S} = 7\mu\text{S}$.

Vidite koliko zbunjujući može biti instrukcijski ciklus instrukcija uslovnog skoka.

Softver programatora

Sećate li se __CONFIG pseudo instrukcije? Za korišćenje WDT, on se mora preko nje uključiti sa WDT ON. A pošto je neki programatori ignorišu, potrebno je i unutar njih podesiti konfiguracione bitove. Postupak njihovog podešavanja razlikuje se od programatora, do programatora (softvera). Uglavnom se traži njihov upis neposredno pre programiranja mikrokontrolera.

15. Interni tajmer

U slučaju da pišete program koji mora imati veliku ali tačno definisanu pauzu sigurno vam se neće svideti mogućnost postavljanja jedne petlje unutar druge i proračunavanja broja ciklusa za željenu dužinu pauze. Postoji elegantniji način rešavanja ovog problema. PIC16F84 ima u sebi integrisan tajmer. Impulsi ovog tajmera mogu biti sinhronizovan sa taktom izvršavanja instrukcija mikrokontrolera ili sa taktom eksternog oscilatora čiji impulsi se dovode na pin 3 (RA4/T0CKL). Glavna funkcija tajmera je brojanje (00h-FFh) koje je za razliku od WDT sinhronizovano sa ovim taktom. U trenutku prekoračenja maksimalne vrednosti, flag tajmera se setuje, i ciklus brojanja opet počinje od nule. Prednost tajmera nad običnom petljom za kašnjenje je u činjenici da je brojanje interni proces mikrokontrolera, i da ni na koji način ne utiče na brzinu izvršavanja glavnog programa. Dovoljno je povremeno proveriti stanje flaga tajmera (ili čekati u praznoj petlji dok flag ne promeni stanje), kako bi mikrokontroler znao da je ciklus brojanja završen.

Kako na ovaj način nije moguće dobiti veće periode, tajmer može koristiti preskaler (već pomenut u poglavlju sa WDT).

Budući da se za korišćenje i WDT i običnog tajmera pretežno vrši manipulacija nad OPTION_REG registrom, vreme je da se bliže upoznate sa njegovom unutrašnjom strukturom.

bit7				bit0			
RBP $\overline{U}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

- $\overline{RBP\overline{U}}$ (eng. Port **B** Pull Up) bit OPTION_REG registra kao što ste već naučili služi za povezivanje internih otpornika PORTB registra na napon napajanja. Crtica iznad njegovog naziva znači da mu je aktivno stanje (uključeni otpornici) pri logičkoj nuli, što se označava kao NOT_RBP $\overline{U}$.
- Upotrebu INTEDG (eng. **I**nterrupt **e**dge) bita naučićete u sledećem poglavlju.
- T0CS (eng. **T**imer**0** **C**lock **S**elect) bit omogućava izbor takta tajmera. Ukoliko je resetovan, brojanje će se odvijati sinhronizovano sa taktom izvršavanja instrukcija koji je četiri puta manji od takta oscilatora. Ovaj takt se može dobiti sa CLKOUT pina mikrokontrolera.

Ukoliko je T0CS bit setovan, brojanje će biti sinhronizovano sa eksternim taktom koji se treba dovesti na pin 3 (RA4/T0CKL). U slučaju odnosa preskalera 1:1, trajanje logičke 1 i logičke 0 eksternog takta mora biti duže od 3 takta oscilatora PIC16F84 (tačnije oko 2 takta + 20nS), a u slučaju drugog odnosa duže od 5 takova (tačnije 4 takta + 40nS).

- T0SE (eng. **T**imer**0** **S**ource **E**dge) bit ima smisla jedino pri upotrebi eksternog takta. Njime se bira hoće li se brojanje vršiti pri prelasku sa logičke 0 na 1 (resetovan T0SE) ili sa logičke 1 na 0 (setovan T0SE).

- PSA (eng. **P**rescaller **A**ssignment) bit se koristi za selekciju pridruženosti preskalera. Ukoliko je setovan, postskalera će biti pridružen WDT, a ukoliko je resetovan preskalera će biti pridružen tajmeru.
- Bitovima PS0 do PS2 bira se odnos deljenja preskalera po sledećoj tablici.

<i>Bit 2, 1, 0</i>	<i>TMR0 odnos</i>	<i>WDT odnos</i>
0, 0, 0	1:2	1:1
0, 0, 1	1:4	1:2
0, 1, 0	1:8	1:4
0, 1, 1	1:16	1:8
1, 0, 0	1:32	1:16
1, 0, 1	1:64	1:32
1, 0, 1	1:128	1:64
1, 1, 1	1:256	1:128

Primećujete da se i ovde pominje WDT odnos. On je tu jer se odnos tajmera od 1:1 može dobiti jedino pridruživanjem preskalera WDT, setovanjem PSA bita. Za takvo podešavanje primenjuje se kod opisan u prošlom poglavlju sa instrukcijama označenim zvezdicama.

Budući da se pri samom programiranju PIC16F84 vrši uključanje ili isključenje WDT (podešavanjem konfiguracionih bitova), ne može se doći u situaciju da se WDT sam uključi, i tako izazove reset mikrokontrolera, iako mu se pridružuje preskalera.

Registar tajmera TMR0 (eng. **T**imer**0**) nalazi se na adresi 01h, a njegov flag T0IF (eng. **T**imer**0** **O**verflow **I**nterrupt **F**lag) nalazi se u INTCON registru (0Bh) na mestu bita 2. Više o ovom registru naučićete u delu sa interaptima.

Sledi program koji ilustruje upotrebu tajmera. On će instrukcijski ciklus preskalerom deliti sa 256, zatim softverski još sa 6, a onda će heksadecimalne brojeve (0 do F) prikazivati na displeju.

Deljenje je kao što se sećate najpraktičnije realizovati rotacijom bita u registru. U programu će biti upotrebljen već ranije ilustrovani princip prikaza na LED displeju.

; ***** Inicijalizacija i imenovanje *****

```
list      p=16F84          ; Definiše upotrebljeni mikrokontroler
#include  <p16F84.inc>      ; Ubacuje nazive registra u program
__CONFIG _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
          ; Podešava konfiguracione bitove
DISP equ  0Ch              ; Promenljiva za broj koji se prikazuje
                              ; na displeju (00h-0Fh)
DEL  equ  0Dh              ; Promenljiva za rotaciju
ORG  00h                   ; Definiše start programa
```

; ***** Inicijalizacija tajmera i displeja *****

```
clrf      TMR0              ; Inicijalizacija tajmera
clrf      INTCON            ; Brisanje flaga tajmera
bsf       STATUS,RP0        ; Prelazak u BANK1
movlw     00h               ; Postavlja sve pinove PORTB
movwf     TRISB              ; kao izlazne
```

```

    movlw    00000111b ; Tajmer je sinhronizovan sa instrukcijskim
                        ; ciklusom, i preskaler je podešen
    movwf    OPTION_REG ; za odnos 1:256
    bcf      STATUS, RP0 ; Povratak u BANK0

; ***** Inicijalizacija brojača *****

Ini  movlw    00h        ; Inicijalizacija
     movwf    DISP       ; displeja
     movlw    00100000b ; Inicijalizacija
     movwf    DEL        ; delitelja

; ***** Čekanje na prekoračenje tajmera *****

Taj  bcf      INTCON, T0IF ; Resetovanje flaga tajmera
Cek  btfss    INTCON, T0IF ; Čekanje na setovanje flaga tajmera,
     goto     Cek         ; odnosno prekoračenje tajmera

; ***** Deljenje sa 6 *****

     rrf      DEL         ; Rotiraj DEL registar nadesno. Kada se Carry
     btfss    STATUS, C   ; flag setuje, nastavi dalje. Ukoliko i dalje nije
     goto     Taj        ; setovan, čekaj na još jedan ciklus tajmera.

; ***** Prikaz cifre na displeju *****

     call     Tabl        ; Uzmi obrazac iz tabele,
     movwf    PORTB       ; i prikaži cifru na displeju.

; ***** Testiranje prekoračenja vrednosti cifre na displeju *****

     movlw    01h        ; 01 u W
     addwf    DISP, F     ; Saberi DISP sa 1. Rezultat u DISP
     btfsc    STATUS, DC  ; Da li je došlo do prekoračenja vrednosti 0Fh?
     goto     Ini         ; Jeste, počni brojanje od početka (cifre 0)
     goto     Taj        ; Nije, nastavi dalje sa brojanjem.

; ***** Tabela *****

Tabl movf     DISP, W     ; U promenljivoj DISP nalazi se vrednost od
     addwf    PCL, 1      ; 00h do 0Fh. Ta vrednost se dodaje na PCL.
     retlw    01111110b ; Obrazac za crtanje cifre 0 - 0
     retlw    00001100b ; Obrazac za crtanje cifre 1 - 1
     retlw    10110110b ; Obrazac za crtanje cifre 2 - 2
     retlw    10011110b ; Obrazac za crtanje cifre 3 - 3
     retlw    11001100b ; Obrazac za crtanje cifre 4 - 4
     retlw    11011010b ; Obrazac za crtanje cifre 5 - 5
     retlw    11111010b ; Obrazac za crtanje cifre 6 - 6

```



```

retlw      00001110b ; Obrazac za crtanje cifre 7 - 7
retlw      11111110b ; Obrazac za crtanje cifre 8 - 8
retlw      11011110b ; Obrazac za crtanje cifre 9 - 9
retlw      11101110b ; Obrazac za crtanje cifre A - A
retlw      11111110b ; Obrazac za crtanje cifre B - B
retlw      01110010b ; Obrazac za crtanje cifre C - C
retlw      01111110b ; Obrazac za crtanje cifre D - D
retlw      11110010b ; Obrazac za crtanje cifre E - E
retlw      11100010b ; Obrazac za crtanje cifre F - F

```

```
; ***** Kraj programa *****
```

```
end ; Kraj programa
```

Možda Vam izgleda čudno upotreba sabiranja umesto inkrementovanja registra DISP. Svakako da se njegovo povećanje moglo uraditi i inkrementovanjem, ali na ovaj način lakše je nakon povećanja testirati prekoračenje njegove maksimalne vrednosti. Jedino instrukcije sabiranja i oduzimanja menjaju stanje DC flaga.

16. Interapti - pojam

Pojam interapta će vam verovatno biti najteži za razumevanje. Ne postoji lak način za njihovo objašnjavanje, ali nadam se da ćete pri kraju ove sekcije moći da ih uspešno primenjujete u svojim programima.

Interapt je proces ili signal koji prekida izvršavanje glavnog mikrokontrolerskog programa i prebacuje njegovo izvršavanje na podprogram koji je zadužen za obradu interapta. Po povratku iz podprograma glavni program nastavlja sa radom kao da se ništa nije ni dogodilo. Ilustrovaću ovo na svakodnevnom primeru. Pretpostavimo da sedite kući i čitate ovo uputstvo. Odjednom vam mobilni signalizira prijem SMS poruke. Prekidate čitanje, pročitate poruku (eventualno i odgovorite), i nastavljate čitanje sa mesta na kome ste stali. Možete zamisliti glavnu rutinu kao čitanje uputstva, zvonjavu mobilnog kao izvor interapta, a proces čitanja poruke (i eventualnog odgovora na nju) kao program za obradu interapta.

PIC16F84 ima 4 izvora interapta. Dva eksterna i dva interna. Za sada ćete naučiti primenu eksternih izvora, a interni će biti objašnjeni kasnije. Ukoliko pogledate oznake izvoda pinova PIC16F84 videćete da je na pinu 6 oznaka RB0/INT. Očigledno je da je RB0 bit 0 PORTB registra. INT označava da se on može konfigurisati i kao pin eksternog interapta. Pri tome se on (samo on) ponaša kao tzv. šmittov okidač (eng. Schmitt trigger), što otežava pojavu lažnih signala okidanja. Takođe se i bitovi 4 do 7 (pinovi 10 do 13) PORTB registra mogu koristiti za interapte. Pre nego što se upotrebi INT ili neki od drugih pinova kao interapt, trebaju se uraditi dve stvari. Prvo se treba reći mikrokontroleru da će se koristiti interapti. Drugo, trebaju se izabrati pinovi koji će se koristiti kao izvori interapta, a ne kao ulazno / izlazni pinovi PORTB registra.

Unutar PIC16F84 nalazi se INTCON (eng. **I**nterrupt **C**ontroller) registar na adresi 0Bh. Njegov bit 7 nazvan je GIE (eng. **G**lobal **I**nterrupt **E**nable). Setovanjem ovog bita mikrokontroler dozvoljava upotrebu jednog ili više izvora interapta.

Bit 4 INTCON registra nazvan je INTE (eng. RB0/INT **I**nterrupt **E**nable) bit „uključi interapt na RB0/INT pinu”. Setovanjem INTE bita u INTCON registru dozvoljena je upotreba pina 6 (RB0/INT PORTB registra) kao izvora interapta. On time gubi funkciju običnog ulazno / izlaznog pina, i dalje se može koristiti kao izvor interapta.

Sada se mikrokontroler treba podesiti za željeni signal izazvanja interapta. Pri rastućoj (0V do +5V) ili pri opadajućoj ivici (+5V do 0V) signala. Drugim rečima da li želite da se interapt u mikrokontroleru izazove pri prelasku signala na pinu 6 sa logičke 0 na logičku 1, ili sa logičke 1 na logičku 0. Ovo se podešava u OPTION_REG registru na adresi 81h. Setovanje njegovog bita 6 nazvanog INTEDG (eng. **I**nterrupt **E**dge) prouzrokuje interapt pri rastućoj ivici, a resetovanje pri silaznoj. Kako je nažalost OPTION_REG registar u BANK1, treba preći iz BANK0 u BANK1, tamo setovati ili resetovati INTEDG bit OPTION_REG registra, i vratiti se u BANK0. Najpraktičnije je ovo podešavanje izvršiti uz ostala u toku inicijalizacije mikrokontrolera.

Ovim je PIC16F84 podešen za korišćenje interapta na RB0/INT pina PORTB registra.

Ukoliko želite koristiti više izvora interapta, možete koristiti bitove 4 do 7 PORTB

registra. Oni se razlikuju od RB0/INT pina u tome što se interapt javlja pri promeni stanja na pinovima. To znači da će se interapt javiti pri prelasku na pinovima 10 do 13 (bitovi 4 do 7 PORTB registra) sa logičke 0 na logičku 1, ali i sa logičke 1 na logičku 0. Podešavanje ovih pinova za izvore interapta vrši se setovanjem bita 3 INTCON registra nazvanog RBIE (eng. Port **B** Change Interrupt **E**nable) „uključiti interapte pri promeni stanja na portu B“. Kako se kod njih interapt izvršava pri promeni stanja pinovi 10 do 13 nemaju svoj bit za određivanje ivice signala pri kojoj će se izazivati interapt.

Interapt flag

Sećate li se kako se vrši setovanje Carry flaga? Pri nailasku na interapt na RB0/INT pinu na sličan način se setuje bit 1 (INTF eng. **I**nterrupt **f**lag), odnosno bit 0 (RBIF eng. Port **B** Interrupt **f**lag) INTCON registra pri interaptu na RB4 do RB7 pinovima. Kada nema interapta, interapt flag je resetovan. To je cela njegova funkcija. Sada se verovatno pitate kakva je njihova svrha. Ukoliko je interapt flag setovan PIC ne može i neće odgovarati na druge interapte. Pretpostavimo da je izazvan interapt. Interapt flag će biti setovan, i mikrokontroler će izvršiti rutinu za obradu interapta. Ukoliko interapt flag nije setovan, PIC će prihvatati nove interapte što može dovesti do neprestanog vraćanja programa na početak rutine za obradu interapta koja se usled toga nikada ne može izvršiti do kraja. Ukoliko se setite primera sa mobilnim, to je kao da Vam za vreme odgovora na SMS stigne nova poruka i prekine upis odgovora. Daleko je praktičnije završiti sa jednom porukom, i onda dozvoliti prijem novih.

Interapt flagovi imaju još jednu funkciju, a to je omogućavanje interapt rutini detekciju izvora interapta (RB0/INT pin odnosno RB4 do RB7 pinovi) u slučaju korišćenja više izvora interapta. Iako PIC automatski setuje interapt flagove pri nailasku interapta, on ih ne resetuje pri povratku iz rutine za obradu interapta. Ovaj posao mora programer sam izvršiti (siguran sam da već znate kako), i treba se odraditi nakon završetka rutine za obradu interapta, a neposredno pre izlaska iz nje.

Memorijske lokacije

U trenutku dovođenja napona napajanja ili prilikom resetu, programski brojač (PC) pokazuje na adresu 00h, na početku programske memorije. Međutim, pri pojavi interapta PC će adresu sledeće instrukcije staviti na stek, i skočiće na adresu 04h. Dakle, kada pišete programe koji će koristiti interapte, pre svega trebate reći mikrokontroleru da preskoči preko adrese 04h, i zadrži interapt rutinu koja počinje na adresi 04h odvojenu od ostatka programa. Ovo se veoma lako implementira.

Najpre je potrebno da program standardno započne pseudoinstrukcijom ORG 00h. Dalje je potrebno preskočiti preko adrese 04h. Ovo se radi postavljanjem GOTO instrukcije, nakon koje sledi labela pozicionirana za start glavnog programa. Nakon ove goto instrukcije sledi još jedna ORG pseudo instrukcija, ovoga puta sa adresom 04h, a nakon nje rutina za obradu interapta. Na kraju programa za obradu interapta potrebno je staviti RETFIE (eng. **R**eturn **f**rom **i**nterrupt) instrukciju. Ova instrukcija označava povratak iz interapt rutine. Kada PIC naiđe na ovu instrukciju PC uzima sa steka lokaciju na kojoj je PIC bio pre nego što se pojavio interapt.

Sledi deo koda koji ilustruje navedeno.

```
org 00h ; PIC počinje odavde pri uključenju i resetu
goto Main ; Idi na glavni program
org 04h ; PIC dolazi ovde pri pojavi interapta
:
: ; Ovde se nalazi rutina za obradu interapta koju
: ; PIC izvršava jedino pri nailasku na interapt.
: ; Na njenom kraju potrebno je obrisati odgovarajući
: ; interapt flag.
:
retfie ; Završetak rutine za obradu interapta

Main ; Početak glavnog programa
```

Postoji par stvari na koje trebate obratiti pažnju prilikom korišćenja interapta.

1. Ukoliko koristite isti registar u glavnom programu i u rutini za obradu interapta imajte u vidu da će se sadržaj tog registra verovatno promeniti pri izvršenju rutine za obradu interapta. Na primer, recimo da koristite W registar da biste poslali podatak na PORTA, i takođe koristite W registar u rutini obradu interapta za neku drugu operaciju. Ukoliko ne pazite W registar će pri povratku iz rutine za obradu interapta zadržati vrednost koju je imao u interapt rutini, i pri povratku iz nje, ovaj podatak biće poslat na PORTA umesto vrednosti koju je W registar imao ranije. Rešenje ovoga je privremeno čuvanje vrednosti W registra u nekom od slobodnih registara, i vraćanje njegove stare vrednosti po završetku interapt rutine. Isto važi i za ostale registre (obično za STATUS) čiji sadržaj se menja u interapt rutini.
2. Potrebno je obratiti pažnju na minimalno vreme između dve uzastopne pojave interapta. Ukoliko se za takt mikrokontrolera koristi kvarc od 4MHz, PIC će podeliti ovu frekvenciju na 4 dela i izvršavati instrukcije taktom od 1MHz. Ovo interno vreme naziva se instrukcijskim ciklusom. U tehničkom uputstvu (eng. Datasheet) za PIC navedeno je da se treba ostaviti najmanje 3 do 4 ciklusa između dve uzastopne pojave interapta. Razlog za kašnjenje je vreme koje je potrebno mikokontroleru za skok na rutinu za obradu interapta, setovanje interapt flaga i povratak iz rutine za obradu interapta. Imajte ovo na umu ukoliko koristite spoljno eelektronsko kolo za izazivanje interapta.
3. Na kraju interapt rutine bit GIE INTCON registra se automatski setuje. Automatsko setovanje GIE bita može da bude i nedostatak u jednom slučaju, tj. moguće je da instrukcija za resetovanje ovog bita (zabranu svih interapta) u glavnom programu uopšte ne bude izvršena. Naime, ukoliko interapt nastupi u trenutku izvršavanja instrukcije koja resetuje GIE bit, prvo će ta instrukcija biti završena do kraja (resetovaće se GIE bit), a zatim će početi izvršavanje interapt rutine koja će na svom kraju da automatski setuje bit GIE. Tako će GIE bit biti setovan iako je neposredno pre interapta bio resetovan. Da bi se zaštitili od ove situacije, Microchip preporučuje da se nakon resetovanja GIE bita proveriti da li je bit zaista resetovan i da se, ukoliko nije, operacija ponovi.

4. Sećate se da se bitovi 4 do 7 na PORTB registru mogu koristiti kao izvori interapta. Ne možete izabrati pojedinačne pinove na PORTB za interapte. Dakle, ukoliko uključite ove pinove, svi postaju dostupni. U čemu je svrha korišćenja 4 bita kao izvora interapta? Primer može biti kućni alarm, čija 4 senzora su povezana na pinove na PORTB registru. Bilo koji senzor može okinuti PIC za uključenje alarma, a rutina za alarm je rutina za obradu interapta. Ovo štedi neprekidno testiranje portova, i rasterećuje mikrokontroler za druge namene. Nažalost njihovim interapt flagom (RBIF INTCON registra) nije moguće izvršiti detekciju interapta na pojedinačnim pinovima, već samo na svim.
5. U slučaju korišćenja više izvora interapta moguće je da se za vreme interapt rutine izazvane RB0/INT pinom pojavi i interapt na bitovima 4-7 PORTB registra. Stoga je potrebno u samoj interapt rutini testirati prvi interapt flag, u zavisnosti od njihovog stanja otići na njegov podprogram, a zatim pre povratka iz interapt rutine testirati i drugi interapt flag, sa eventualnim skokom i na njegov podprogram.
6. Interapt flagovi setuju se bez obzira na stanje GIE bita, kao i pojedinačnih dozvola za interapte (kao u programu sa tajmerom). To može biti iskorišćeno za detekciju njihovih stanja iz samog programa bez korišćenja interapta. Naravno, odgovarajući interapt flag je kao i kod interapta potrebno ručno resetovati.

U sledećem poglavlju videćete program koji koristi interapte.

17. Interapti - program

Prešli ste dosta teorije u prošlom poglavlju, i došlo je vreme da se upoznate sa programom koji koristi interapte. Program će pri pritiskanju prekidača brojiti od 0 do 7. Rezultat će se prikazivati na 3 LED na PORTA u binarnom obliku. Sam glavni program će samo prikazivati rezultat, a brojanje i testiranje prekoračenja biće realizovani interapt rutinom.

Pre samog programa upoznajte CLRf instrukciju. Ona postavlja vrednost 00h u željeni registar. Pri tome se u STATUS registru setuje Zero flag.

Najpre se mikrokontroler treba podesiti tako da pri izvršavanju programa preskoči deo na koji PC skače pri pojavi interapta.

```
org      00h      ; Ovde PC dolazi pri uključenju i resetu
goto     Main     ; Odlazak na glavni program

org      04h      ; Ovde će početi rutina za obradu interapta
;
;               ; Rutina za obradu interapta. Upisaće se kasnije
;
retfie    ; Ova instrukcija označava kraj rutine za obradu
           ; interapta, i vraća PC na glavni program.
```

Main ; Početak glavnog programa

Onda je potrebno podesiti PIC za korišćenje interapta na pinu 6:

```
bsf      INTCON, GIE ; Global interrupt enable (1 – uključi)
bsf      INTCON, INTE ; RB0 Interrupt enable (1 – uključi)
```

Za svaki slučaj potrebno je resetovati interapt flag (nikad ne veruj mikrokontroleru):

```
bcf      INTCON, INTF ; Obriši interapt flag
```

Kako su ovde iskorišćene čak 3 instrukcije za menjanje vrednosti samo jednog registra praktičnije je u pogledu štednje memorije i brzine izvršavanja programa zameniti ih sa dve.

```
movlw    10010000b ; bit 7 GIE – Global interrupt enable (1 – uključi)
                        ; bit 4 INTE – RB0 Interrupt enable (1 – uključi)
                        ; bit 1 INTF – Interrupt flag (0 – obriši)
movwf    INTCON      ; Inicijalizuj INTCON registar
```

Sada je potrebno podesiti portove. Sećate se da zbog korišćenja RB0 kao interapt pina, on mora biti postavljen kao ulazni;

```

bsf          STATUS, RP0      ; Prebacuje nas u BANK1
movlw        01h             ; 00000001b u TRISB
movwf        TRISB           ; RB0/INT kao ulaz, a ostali kao izlaz
clrf         TRISA            ; Svi pinovi na PORTA su izlazni
bcf          OPTION_REG, INTEDG ; Interapt se izaziva pri
                                ; opadajućoj ivici signala
bcf          OPTION_REG, NOT_RBPU ; Uključeni interni „pull up“
                                ; otpornici na PORTB
bcf          STATUS, RP0      ; Povratak u BANK0

```

Kako interapt želimo izazivati pri opadajućoj ivici signala (sa +5V na 0V) mora se podesiti i INTEDG bit OPTION_REG registra. Takođe je resetovanjem bita RBPU bita OPTION_REG registra interno zadano spajanje svih ulaza na PORTB za otpornike prema naponu napajanja.

Program će koristiti promenljivu TEMPW za privremeno čuvanje W registra dok se izvršava interapt rutina i promenljivu BROJAC za pamćenje broja pritisnutih prekidača. Vrednost promenljive BROJAC neće biti 0-7, već 0-28 u koracima po 4 zbog lakšeg dovođenja na pinove RA3, RA4 i RA5. BROJAC će u programu imati sledeće vrednosti:

bitovi	7	6	5	4	3	2	1	0	
	0	0	0	0	0	0	0	0	00h
	0	0	0	0	0	1	0	0	04h
	0	0	0	0	1	0	0	0	08h
	0	0	0	0	1	1	0	0	0Ch
	0	0	0	1	0	0	0	0	10h
	0	0	0	1	0	1	0	0	14h
	0	0	0	1	1	0	0	0	18h
	0	0	0	1	1	1	0	0	1Ch
	0	0	1	0	0	0	0	0	20h - prekoračenje

Stoga je najpre potrebno pridružiti slobodne registre promenljivim, i inicijalizovati vrednost BROJAC registra.

```

TEMPW      equ 0Ch      ; TEMPW na adresi 0Ch
BROJAC     equ 0Dh      ; BROJAC na adresi 0Dh
clrf      BROJAC       ; Stavi 00h u BROJAC

```

Dalje je potrebno u glavnom programu napraviti petlju koja će vrednost promenljive BROJAC neprestano prikazivati na PORTA registru.

```

Pri movf    BROJAC, W    ; BROJAC u W registar
    movwf   PORTA        ; a odatle u PORTA
    goto    Pri          ; Vрати se na početak petlje
    end                      ; Kraj programa

```

Budući da prva pojava interapta nastaje tek kada se program nađe u glavnoj programskoj petlji (Pri), i da u sama petlja ne reaguje na stanje bitova STATUS registra, u interapt rutini nema potrebe za privremenim čuvanjem njegove vrednosti.

U interapt rutini potrebno je najpre sačuvati vrednost W registra.

```
movwf    TEMPW    ; Privremeno čuvanje sadržaja W registra
```

Onda je potrebno uvećati sadržaj BROJAC registra za 4. To se najlakše može uraditi sabiranjem sa 4.

```
movlw    04h      ; 04h (korak brojanja) u W registar
addwf    BROJAC, F ; Saberi 04h sa BROJAC.
```

Dalje je potrebno proveriti da li je promenljiva BROJAC prekoračila maksimalnu vrednost 1Ch (00011100b) i došla do 20h - 00100000b. Najpraktičnije je izvršiti testiranje petog bita BROJAC registra.

```
btfsc    BROJAC, 5 ; Testiraj peti bit 00100000 BROJAC registra.
           ; Ukoliko nije setovan, preskoči sledeću
           ; instrukciju
clrf     BROJAC    ; Upisi 00h u BROJAC
```

```
           ; Ukoliko je BROJAC nakon povećanja za 4
           ; prekoračio maksimalnu vrednost (1Ch)
           ; sada je jednak 00h. Ukoliko
           ; nije, sada je uvećan za 4.
```

```
movf     TEMPW, W  ; Vрати prethodni sadržaj W registra
bcf      INTCON, INTF ; Obriši INTF – dozvoli nove interapte
retfie                   ; Kraj interapt rutine.
```

Pogledajte sada celokupan program:

```
; ***** Inicijalizacija i imenovanje *****
```

```
list      p=16F84      ; Definiše upotrebljeni mikrokontroler
#include   <p16F84.inc>  ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
           ; Podešava konfiguracione bitove
ORG       00h          ; Definiše start programa
TEMPW     equ 0Ch       ; Čuvanje sadržaja W registra na adresi 0Ch
BROJAC     equ 0Dh       ; BROJAC na adresi 0Dh
```

```
; ***** Interapt rutina *****
```

```
org       00h          ; Ovde PC dolazi pri uključenju i resetu
goto      Main          ; Odlazak na glavni program

org       04h          ; Ovde će početi rutina za obradu interapta
movwf     TEMPW         ; Privremeno čuvanje sadržaja W registra
movlw     04h           ; 04h (korak brojanja) u W registar
addwf     BROJAC, F      ; Saberi 04h sa BROJAC.
           ; Rezultat u BROJAC registar
```



```

    btfsc      BROJAC, 5 ; Testiraj peti bit 00100000 BROJAC registra.
                    ; Ukoliko nije setovan, preskoči sledeću
                    ; instrukciju
    clrf      BROJAC ; Upisi 00h u BROJAC
                    ; Ukoliko je BROJAC nakon povećanja za 4
                    ; prekoračio maksimalnu vrednost (1Ch) sada je
                    ; jednak 00h. Ukoliko nije, sada je uvećan za 4.
    movf      TEMPW, W ; Vрати prethodni sadržaj W registra
    bcf       INTCON, INTF ; Obriši INTF – dozvoli nove interapte
    retfie    ; Kraj interapt rutine.

; ***** Glavni program – inicijalizacija *****

Main movlw    10010000b ; bit 7 GIE – Global interrupt enable (1 – uključi)
                    ; bit 4 INTE – RB0 Interrupt enable (1 – uključi)
                    ; bit 1 INTF – Interrupt flag (0 – obriši)
    movwf     INTCON ; Inicijalizuj INTCON registar
    bsf      STATUS, RP0 ; Prebacuje nas u BANK1
    movlw    01h ; 00000001b u TRISB
    movwf    TRISB ; RB0/INT kao ulaz, a ostali kao izlaz
    movlw    00h ; 000000b u TRISA
    movwf    TRISA ; svi pinovi na PORTA su izlazni
    bcf      OPTION_REG, INTEDG ; Interapt se izaziva pri
                    ; opadajućoj ivici signala
    bcf      OPTION_REG, NOT_RBPU ; Uključeni interni „pull up“
                    ; otpornici na PORTB
    bcf      STATUS, RP0 ; Povratak u BANK0
    clrf     BROJAC ; Stavi 00h u BROJAC

; ***** Glavni program – petlja *****

Pri movf      BROJAC, W ; BROJAC u W registar
    movwf     PORTA ; a odatle u PORTA
    goto      Pri ; Vрати se na početak petlje

; ***** Kraj programa *****

end ; Kraj programa.

```

Električna šema za program sastoji se od LED1, LED2 i LED3 sa svojim otpornicima od 100Ω, prekidača T3 sa kondenzatorom od 1nF i RC oscilatora od 330kΩ i 220pF. Kondenzator kod prekidača je u ovoj šemi neophodan zbog varničenja kontakta. Bez njega bi se jednim pritiskom prekidača izazvalo 2-3 interapta.

Sigurno ste primetili neuobičajen način povezivanja LED3. Ona se mora tako povezati, jer RA4 pin ne može na svom izlazu dati logičku jedinicu. U slučaju da se na njega pošalje logička 0, ponašaće se isto kao i ostali pinovi, a u slučaju jedinice, preći će u stanje visoke impedanse (kao da nije ni povezan). U ovom primeru, pri logičkoj nuli, struja će prolaziti kroz otpornik i PIC, a pri logičkoj 1 kroz otpornik i

LED. U pravim projektima ovo se rešava povezivanjem LED sa otpornikom prema naponu napajanja, umesto prema masi. Pri takvom povezivanju LED će svetleti pri dovođenju logičke nule na njoj odgovarajući pin.

Pin RA4 razlikuje se od svih ostalih i po svom ulazu. Ulaz mu je sa tzv. Šmitovim okidačem (eng. Schmitt Trigger). To omogućava bolji prijem signala sa prisustvom šuma (npr. sa fotootpornika ili udaljenog senzora temperature).

Do sada ste naučili 31 instrukciju. Verovatno i sami uočavate da Vam više ne predstavlja problem učenje novih instrukcija koliko funkcije pojedinih registara.

U registare koje ste u potpunosti naučili spadaju PORTA, TRISA, PORTB, TRISB, PCL, PCLATH, TMR0 i OPTION_REG. To je osam potpuno i dva delimično (STATUS i INTCON) naučena registra.

Sledi još jedan deo sa interaptima, a onda nešto novo.

18. Tajmerom izazvani interapti

Sećate li se tajmera. Malo je nepraktično sve vreme čekati na promenu njegovog flaga. Praktičnije bi bilo da mikrokontroler za vreme brojanja tajmera obavlja neku drugu operaciju a da se pri setovanju njegovog flaga izvrši interapt rutina. Pogađate, moguće je.

Pre nego što naučite kako se to može ostvariti, upoznaćete se sa internom strukturom INTCON registra (0Bh).

bit7				bit0			
GIE	EEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF

- GIE (eng. **G**lobal **I**nterrupt **E**nable) omogućava ili zabranjuje upotrebu svih interapta. Njegovim setovanjem, dozvoljena je upotreba interapta.
- Upotrebu EEIE bita naučićete u delu sa upotrebom EEPROM memorije.
- Setovanjem TOIE (eng. **T**imer**0** **I**nterrupt **E**nable) bita tajmer je upotrebljen kao izvor interapta.
- INTE (eng. **R**B0/**I**NT **I**NTerrupt **E**nable) bit setovanjem dozvoljava korišćenje pina 6 mikrokontrolera kao izvora interapta.
- Setovanjem RBIE (eng. **P**ort **B** **C**hange **I**nterrupt **E**nable) bita, omogućena je upotreba PORTB registra kao izvora interapta pri promeni logičkog stanja na pinovima 10 do 13 mikrokontrolera.
- TOIF (eng. **T**imer**0** **I**nterrupt **F**lag) bit setuje se pri prekoračenju tajmera.
- INTF (eng. **I**nterrupt **F**lag) bit setuje se pri dolasku odgovarajućeg logičkog nivoa (odabranog INTEDG bitom OPTION_REG registra) na pin 6 (RB0/INT) mikrokontrolera.
- RBIF (eng. **P**ort **B** **I**nterrupt **f**lag) setuje se pri promeni logičkog stanja na pinovima 10 do 13 mikrokontrolera.

Nijedan interapt flag se ne resetuje automatski nakon očitavanja. Da bi se dozvolili novi interapti, neophodno je resetovati željeni flag pre izlaska iz interapt rutine.

Da bi se tajmer upotrebio kao izvor interapta, dovoljno je dozvoliti njegovo korišćenje setovanjem GIE i TOIE bitova, i resetovanjem njegovog flaga. Sva podešavanja vezana za običan tajmer (preskaler, eksterni takt) važe i u ovom slučaju.

Sledi program koji ilustruje upotrebu 3 nezavisna dela. Prvim delom će LED1 treperiti. To će biti glavni program. Drugim delom će se pri pritisku na T3 izazvati interapt koji će promeniti stanje na LED2. Trećim delom će se tajmerom izazivati interapt koji će menjati stanje na LED3.

Ova tri dela su samo uslovno nezavisna. Interapt rutina troši vreme potrebno za izvršavanje svojih instrukcija, pa će u zavisnosti od brzine interapt rutine (i brzine njenog pojavljivanja) glavni program raditi sporije. Međutim, interapt rutina uvek se pravi tako da njeno trajanje bude što kraće, pa njeno izvršavanje ne utiče u velikoj meri na brzinu glavnog programa.

; ***** Inicijalizacija i imenovanje *****

```
list      p=16F84          ; Definiše upotrebljeni mikrokontroler
#include  <p16F84.inc>      ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                                ; Podešava konfiguracione bitove
ORG      00h                ; Definiše start programa
TEMPW     equ 0Ch           ; Čuvanje sadržaja W registra na adresi 0Ch
TEMPSTATUS equ 0Dh         ; Čuvanje sadržaja STATUS registra na 0Dh
BROJAC1    equ 0Eh         ; BROJAC1 na adresi 0Eh. Inicijalno FFh
BROJAC2    equ 0Fh         ; BROJAC2 na adresi 0Fh. Inicijalno FFh
BROJAC3    equ 10h        ; BROJAC3 na adresi 10h. Inicijalno FFh
```

; ***** Interapt rutina *****

```
org      00h                ; Ovde PC dolazi pri uključenju i resetu
goto     Main               ; Odlazak na glavni program

org      04h                ; Ovde će početi rutina za obradu interapta
movwf    TEMPW              ; Čuvanje sadržaja W registra u TEMPW
swapf    STATUS, W          ; STATUS sa okrenutim niblovima u W
movwf    TEMPSTATUS         ; i zatim u TEMPSTATUS registar
```

; ***** Test interapta izazvanog prekidačem *****

```
btfss    INTCON, INTF       ; Interapt izazvan prekidačem?
goto     Taj                 ; Nije, idi na deo za tajmerski interapt.
```

; ***** Rutina za obradu interapta izazvanog prekidačem *****

```
movlw    00001000b          ; Maska za LED2
xorwf    PORTA, F            ; Promeni stanje LED2
goto     Kraj                ; Izadi iz interapt rutine
```

; ***** Rutina za obradu interapta izazvanog tajmerom *****

```
Taj      decfsz    BROJAC3, f ; Smanji BROJAC3.
goto     Kraj            ; Ako još nije 0, izadi iz interapt rutine
movlw    00010000b       ; Stigao je do 0. Maska za LED3
xorwf    PORTA, F        ; Promeni stanje LED3
```

; ***** Izlazak iz interapt rutine *****

```
Kraj  swapf    TEMPSTATUS, W    ; TEMPSTATUS sa okrenutim niblovima
                                     ; u W. Dva puta okrenuti niblovi daju
      movwf    STATUS            ; prvobitno stanje koje ide u STATUS
      swapf    TEMPW, F          ; Jednom okreni niblove u samom
                                     ; TEMPW registru,
      swapf    TEMPW, W          ; a drugi put sa W kao odredištem.
      bcf      INTCON, T0IF      ; Dozvoli nove interapte tajmera
      bcf      INTCON, INTF      ; Dozvoli nove interapte prekidača
      retfie                               ; Kraj interapt rutine.
```

; ***** Glavni program – inicijalizacija *****

```
Main  clrf     TMR0              ; Inicijalizacija tajmera

      movlw    10110000b ; bit 7 GIE – Global interrupt enable (1 – uključi)
                                     ; bit 4 INTE – RB0 Interrupt enable (1 – uključi)
                                     ; bit 1 INTF – Interrupt flag (0 – obriši)
                                     ; bit 5 T0IE – Timer interrupt enable (1 – uključi)
                                     ; bit 2 T0IF – Timer interrupt flag (0 – obriši)
      movwf    INTCON            ; Inicijalizuj INTCON registar

      bsf      STATUS, RP0       ; Prebacuje nas u BANK1

      movlw    00000111b ; T0CS=0 - Tajmer je sinhronizovan sa
                                     ; instrukcijskim ciklusom
                                     ; Preskaler - PSA=111 je podešen za odnos 1:256
                                     ; INTEDG=0 - Interapt se izaziva pri
                                     ; opadajućoj ivici signala (sa 1 na 0)
                                     ; NOT_RBPU=0 - Uključeni interni „pull up“
                                     ; otpornici na PORTB
      movwf    OPTION_REG

      movlw    01h              ; 00000001b u TRISB
      movwf    TRISB            ; RB0/INT kao ulaz, a ostali kao izlaz
      movlw    00h              ; 00000b u TRISA
      movwf    TRISA            ; svi pinovi na PORTA su izlazni
      bcf      STATUS, RP0      ; Povratak u BANK0
```

; ***** Glavni program – treptanje LED1 *****

```
Poc   movlw    00000100b ; Maska za LED1
      xorwf    PORTA, F    ; Promeni stanje LED1
      call     Pet          ; Ubaci kašnjenje
      goto     Poc          ; Vрати se na početak
```

```

; ***** Podprogram za kašnjenje *****

Pet  decfsz    BROJAC1, F ; Ove dve petlje služe za brojanje  nadole od
goto          Pet          ; 255 do 0, 255 puta, omogućavajući nam da
decfsz    BROJAC2, F ; možemo videti kako LED1 treperi
goto          Pet
return                                ; Povratak iz podprograma

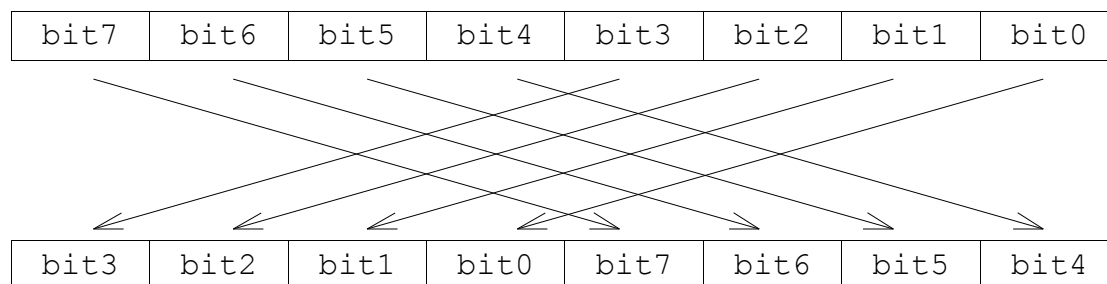
; ***** Kraj programa *****

                                end                ; Kraj programa

```

Možete primetiti da je u ovu interapt rutinu ubačeno privremeno čuvanje STATUS registra, budući da se njegov sadržaj (tačnije Zero flag) u interapt rutini može promeniti xorwf instrukcijom. Najlakše je realizovati njegovo čuvanje koristeći SWAPF instrukciju.

Swapf (eng. **Swap F**) „zameni“ instrukcija menja mesta gornjem i donjem niblu u registru.



Sintaksa instrukcije je swapf f,d pri čemu je f registar nad kojim se vrši operacija, a sa d je određeno određište rezultata operacije.

Možete se zapitati zašto su upotrebljene dve swapf instrukcije pri vraćanju TEMPW u W, kada se isto moglo realizovati i movf TEMPW,W instrukcijom. Razlog je taj što će movf instrukcija setovati Zero flag u slučaju da je u W bila vrednost 00h, dok je swapf instrukcija neutralna prema svim flagovima. Slično važi i za STATUS registar. Ukoliko je određište instrukcije koja menja stanje flagova STATUS registra sam STATUS registar, njegovi flagovi postaviće se u zavisnosti od rezultata instrukcije, bez obzira na stanje bajta koji želimo poslati u njega.

U delu sa testiranjem interapta možete primetiti da je testiran samo interapt flag prekidača. To je sasvim normalno, jer nešto mora da je izazvalo interapt kada se već ušlo u interapt rutinu. Ukoliko to nije bio prekidač, onda je sigurno bio tajmer.

Sigurno ste u delu za obradu tajmerskog interapta uočili neobičnu realizaciju kašnjenja. Ono se moralo izvesti ovako (pri prolasku samo uvećanje bez čekanja), jer bi da je izvedeno na klasičan način, rutina za obradu interapta trajala previše dugo, što bi prouzrokovalo preveliko kašnjenje glavnog programa.

Dosta je bilo interapta. Sledi nešto sasvim drugačije.

19. Indirektno adresiranje

Sećate li se tabele? Kod nje je skok na željenu lokaciju izvršen sabiranjem PCL sa W. Nešto slično ali u odnosu na registre, moguće je postići korišćenjem FSR i INDF registra.

INDF registar nije fizički registar. Njegovo adresiranje u stvari adresira registar čija adresa je sadržana u FSR registru (FSR je pointer adrese željenog registra). Taj postupak naziva se indirektno adresiranje.

Recimo da PORTA sadrži vrednost 03h, a PORTB 2Bh. Ukoliko u FSR ubacite adresu PORTA registra (05h), i pročitate sadržaj registra INDF, dobićete vrednost 03h. Ukoliko sada inkrementujete FSR, on će ukazivati na PORTB (06h). Sada će iščitavanje vrednosti iz INDF registra vratiti vrednost 2Bh.

Isto tako je moguće i upisivanje podataka u željene registre.

Sledeći program ilustruje upisivanje vrednosti 00h u registre od 20h do 2Fh.

```
        movlw      00100000b ; Inicijalizacija pointera
        movwf      FSR        ; na registar 20h
Pet     clrff      INDF       ; Indirektno brisanje željenog registra
        incf       FSR        ; Pozicioniranje pointera na sledeći registar
        btfss      FSR, 4     ; Prekoračenje? 30h – 00110000b
        goto       Pet        ; Nije, nastavi sa brisanjem
```

; Jeste, nastavak programa.

Za razliku od tabele, zbog malog broja registara kod upotrebe indirektnog adresiranja nema ograničenja u smislu blokova podataka.

Indirektno adresiranje se u programima upotrebljava jedino u slučaju veće količine podataka.

Do sada ste naučili 32 od 35 instrukcija, potpunu upotrebu deset registra i delimičnu upotrebu dva registra od ukupno 15. Prešli ste najveći (i najteži – interapti) deo gradiva, međutim ima se još mnogo toga naučiti.

20. EEPROM memorija

Zamislite da imate PIC koji preko sistema fotoćelija broji posetioce koji su ušli kroz ulazna vrata banke, kao i one koji su izašli kroz izlazna vrata, i na osnovu broja lica u banci reguliše rad senzora pokreta, osvetljenja i klima uređaja.

Kako ne biste želeli da išta prepustite slučaju, mikrokontroler radi sa uključenim WDT. Međutim, elektrodistribucija se postara da ne bude sve tako idealno i odjednom nestane struje. Kako banka ne bi bila banka ukoliko nema neku rezervu, 2 sekunde nakon nestanka struje uključuje se elektroagregat (što ne sprečava reset kompjutera i zabrinuto „Jao dođite kasnije, ne rade nam terminali“), i PIC (koji se takođe resetovao) kreće sa radom inicijalizujući u svom brojaču 1 osobu (u normalnom radu onoga ko prvi dolazi na posao). Nakon što se broj osoba u banci smanji za 1 osobu (što može nastati i mnogo kasnije, jer su klijenti banke čuli da je legla plata), prestaju da rade klima uređaji i osvetljenje, a senzori pokreta uključuju alarm i dojavljuju pljačku banke policiji.

Znam znam. Jedan manji akumulator ili UPS rešio bi problem. Ali direktor banke ni da čuje. To direktno poskupljuje projekat. Pa nije valjda džabe kupio agregat!

U ovom slučaju potrebno je da PIC i nakon nestanka napona napajanja zapamti vrednost promenljive BROJAC (ili više promenljivih ukoliko je broj osoba koje mogu stati u prostoriju veći od 255). Po dolasku napona napajanja ove vrednosti trebaju se pročitati, i prebaciti u odgovarajuće registre.

Unutar PIC16F84 nalazi se 64 bajta EEPROM memorije (00h–3Fh). Za razliku od registara koji su obična RAM memorija, ova memorija zadržava svoje stanje i nakon nestanka napona napajanja mikrokontrolera.

Nažalost zbog fizičkih osobina EEPROM memorije rad sa njom nije ni izdaleka tako jednostavan kao rad sa običnim RAM registrima. Pored indirektnog adresiranja (za šta se koriste registri EEADR (09h eng. **EEPROM Adress**) i EEDATA (08h eng. **EEPROM Data**)) potrebno je inicirati proces čitanja ili snimanja bajta, što se vrši preko EECON1 (88h eng. **EEPROM Controller1**) i EECON2 (89h eng. **EEPROM Controller2**) registra.

Pre nego što naučite čitanje i snimanje u EEPROM, upoznaćete se sa internom strukturom EECON1 registra (88h).

bit7								bit0
/	/	/	EEIF	WRERR	WREN	WR		RD

- Bitovi 5 do 7 se ne koriste u EECON1 registru.
- EEIF (eng. **EEPROM Interrupt Flag**) bit predstavlja flag interapta izazvanog završetkom procesa snimanja bajta. Ovaj interapt može se dozvoliti setovanjem EEIE (eng. **EEPROM Interrupt Enable**) bita INTCON registra. Kao i bilo koji drugi interapt flag, potrebno ga je ručno obrisati pre povratka iz

interrupt rutine.

- **WRERR** (eng. **Write Error**) bit indikuje prekid snimanja bajta u EEPROM, usled isteka vremena WDT ili eksternog reseta mikrokontrolera. Program može nakon reseta testirati ovaj bit (uz to mora otkriti i da li je reset nastao usled nestanka napona napajanja ili ne), i na osnovu njegovog stanja dovršiti proces snimanja (sadržaj EEADR i EEDATA registra ne menja se nakon reseta).
- **WREN** (eng. **Write Enable**) bit mora biti setovan, da bi bio moguć upis u EEPROM.
- **WR** (eng. **Write**) bit koristi se za iniciranje upisa u EEPROM memoriju.
- **RD** (eng. **Read**) bit koristi se za iniciranje čitanja iz EEPROM memorije.

Pogledajte najpre primer čitanja memorijske adrese 10h iz EEPROM memorije.

```
bcf      STATUS, RP0      ; Prebacuje nas u BANK0 zbog EEADR
movlw    10h              ; Želimo pročitati sadržaj EEPROM
movwf    EEADR            ; memorije sa adrese 10h
bsf      STATUS, PR0      ; Prebacuje nas u BANK1
bsf      EECON1, RD       ; Inicira proces čitanja
                          ; EEPROM memorije

bcf      STATUS, RP0      ; Prebacuje nas u BANK0
movf     EEDATA, W        ; Sadržaj bajta sa adrese 10h
                          ; iz EEPROM memorije je u EEDATA,
                          ; odakle se može prebaciti u W
```

Ovaj program trebao bi da Vam je u potpunosti jasan. Setovanjem RD bita iniciran je proces čitanja, a zatim se RD bit automatski nakon čitanja vraća na 0. Podatak iz EEPROM memorije dostupan je u EEDATA registru, ali samo u toku narednog instrukcijskog ciklusa. Zato je odmah premešten u W registar. Pogledajte sada kako je realizovano snimanje bajta 24h u EEPROM adresu 10h.

```
bcf      STATUS, RP0      ; Prebacuje nas u BANK0
movlw    24h              ; Bajt za snimanje
movwf    EEDATA           ; u EEDATA
movlw    10h              ; na adresu
movwf    EEADR            ; 10h
bsf      STATUS, RP0      ; Prebacuje nas u BANK1 zbog EECON1
bcf      INTCON, GIE      ; Zabrana svih interapta (ukoliko ih ima)
bsf      EECON1, WREN     ; Dozvola pisanja u EEPROM

movlw    55h              ;
movwf    EECON2           ;
movlw    AAh              ; Inicijalizacija upisa u EEPROM
movwf    EECON2           ;
bsf      EECON1, WR       ;
```

bsf	INTCON, GIE	; Dozvola interapta (ukoliko se koriste)
bcf	EECON1, WREN	; Zabrana daljeg pisanja u EEPROM

Snimanje u EEPROM neće se inicirati ukoliko nije tačno ispoštovana procedura (55h u EECON2, AAh u EECON2, setovan WR) i to bez pauza (otud zabrana interapta) za svaki pojedinačni bajt. Microchip navodi da je ta procedura uvedena kako bi se sprečio nehotičan upis u EEPROM zbog mogućih grešaka u programu. Iz istog razloga WREN bit je potrebno držati setovanim jedino za vreme upisa u EEPROM.

WR bit se automatski resetuje po završetku upisa. U slučaju serijskog snimanja podataka u EEPROM, potrebno je pre sledećeg snimanja proveriti stanje ovog bita, kako se ne bi desilo iniciranje novog upisa u EEPROM a da prethodno snimanje još nije završeno. To je obavezno, jer snimanje u EEPROM nije sinhronizovano sa instrukcijskim ciklusom, već se samo za EEPROM koristi poseban interni oscilator.

Možda Vam se mere inicijalne procedure i WRERR bita čine preteranim. Ali zapitajte se šta bi se desilo ukoliko bi u EEPROM memoriji čuvali npr. obrasce za prikaz broja na LED displeju. U slučaju korupcije EEPROM memorije PIC ni nakon resetu ne bi mogao nastaviti sa normalnim radom. Morao bi se odlemiti sa štampane pločice i ponovo programirati (u mikrokontroler se sa programom snima i sadržaj EEPROM memorije).

U sledećem poglavlju upoznaćete se sa programom koji koristi EEPROM memoriju i naučićete upotrebu EEIF bita i proces upisa podataka u EEPROM iz samog assemblera. Na taj način se sadržaj EEPROM memorije može čuvati zajedno sa programom u jednom .asm ili .hex fajlu.

21. EEPROM i interapti

Sećate li se interapta (kao da biste ih mogli zaboraviti)? U njihovim osnovnim podacima navedeno je da se mogu koristiti 4 izvora interapta. Dva hardverska (željena promena stanja na INT pinu i promena stanja na pinovima RB4-RB7) i dva softverska (prekoračenje tajmera i završetak slanja podatka u EEPROM).

Ima li svrhe koristiti završetak slanja podatka u EEPROM kao izvor interapta? Teško. Sav proces može se završiti linijskim kodom. Opravdanje njenog korišćenja može se naći u vremenski kritičnim programima kod kojih je potrebno za vreme snimanja u EEPROM izvršavati druge operacije. Bolje je da program za vreme čekanja završetka upisa u EEPROM radi nešto korisno.

Sada ćete se upoznati sa programom koji pritiskom na prekidač T3 menja stanje na LED displeju od 9 do 0, i trenutno stanje na njemu snima u EEPROM. Po isključenju napona napajanja, i njegovom ponovnom dovođenju, na LED displeju naći će se poslednja zapamćena vrednost (to je i bio cilj u primeru sa bankom). Glavni program imaće jedino funkciju učitavanja i snimanja trenutnih vrednosti u EEPROM, a interapt rutinom će se realizovati sve ostalo (test prekidača, smanjenje promenljive, prikaz na displeju). Kod ovakvog programa očigledno je da interapt rutina traje daleko duže od samog glavnog programa. Kako glavni program nije mnogo opterećen, to se ovde može tolerisati.

; ***** Inicijalizacija i imenovanje *****

```
list      p=16F84      ; Definiše upotrebljeni mikrokontroler
#include  <p16F84.inc>  ; Ubacuje nazive registra u program
__CONFIG _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
          ; Podešava konfiguracione bitove
TEMPW     equ  0Ch      ; Promenljiva za W registar
TEMPSTATUS equ  0Dh      ; Promenljiva za STATUS registar
DISP      equ  0Eh      ; Promenljiva za broj koji se prikazuje
          ; na displeju i snima (09h – 00h)
```

; ***** Inicijalizacija vrednosti u EEPROM memoriji *****

```
org      2100h      ; Početak EEPROM memorije
de       00h        ; Inicijalizacija početne vrednosti na
          ; displeju
```

; ***** Interapt rutina – čuvanje sadržaja registra *****

```
org      00h        ; Ovde PC dolazi pri uključenju i resetu
goto     Main        ; Odlazak na glavni program
org      04h        ; Ovde će početi interapt rutina

movwf    TEMPW        ; Čuvanje sadržaja W registra u TEMPW
swapf    STATUS, W    ; STATUS sa okrenutim niblovima u W
movwf    TEMPSTATUS   ; i zatim u TEMPSTATUS registar
```

; ***** Test prekidača *****

```
        bcf      STATUS,RP0      ; Povratak u BANK0 – u glavnom
                                   ; programu pre skoka na interapt rutinu
                                   ; program se izvršavao u BANK1
Cek1    btfss    PORTB,0          ; Testiranje otpuštenosti prekidača
        goto     Cek1            ; Nije otpušten
Cek2    btfss    PORTB,0          ; Još jedno testiranje zbog imunosti na
        goto     Cek2            ; eventualna varničenja kontakta.
Cek3    btfsc    PORTB,0          ; Testiranje pritisnutosti prekidača
        goto     Cek3            ; Nije pritisnut
Cek4    btfsc    PORTB,0          ; Još jedno testiranje zbog imunosti na
        goto     Cek4            ; eventualna varničenja kontakta.
```

; ***** Prikaz cifre na displeju *****

```
        movf     DISP,W          ; DISP u W
        call     Tab1            ; Uzmi obrazac iz tabele,
        movwf    PORTB           ; i prikaži cifru na displeju.
```

; ***** Testiranje prekoračenja vrednosti cifre na displeju *****

```
        decf     DISP,F          ; Smanji DISP za 1. Rezultat u DISP
        btfss    DISP,7          ; Testiraj bit 7. On će biti 1 jedino ukoliko je
                                   ; došlo do prekoračenja sa 00h na FFh
        goto     Povr            ; Nije, izađi iz interapt rutine
        movlw    09h             ; Jeste, inicijalizuj brojac na
        movwf    DISP            ; vrednost 09h

Povr    swapf    TEMPSTATUS,W     ; TEMPSTATUS sa okrenutim niblovima
                                   ; u W. Dva puta okrenuti niblovi daju
        movwf    STATUS           ; prvobitno stanje koje ide u STATUS
                                   ; Sada je program u BANK1
        swapf    TEMPW,F          ; Jednom okreni niblove u samom
                                   ; TEMPW registru,
        swapf    TEMPW,W          ; a drugi put sa W kao odredištem.
        bcf      EECON1,EEIF      ; Dozvola novih interapta
        retfie                   ; Kraj interapt rutine.
```

; ***** Glavni program – inicijalizacija *****

```
Main    bsf      STATUS,RP0      ; Prebacuje nas u BANK1
        bcf      EECON1,EEIF      ; Reset interapt flaga
        movlw    11000000b        ; bit 7 GIE=1 – Global interrupt enable
        movwf    INTCON           ; bit 6 EEIE=1 – EEPROM interrupt enable
        movlw    00000000b        ; NOT_RBPU=0 - Uključeni interni „pull up“
        movwf    OPTION_REG       ; otpornici na PORTB

        movlw    01h              ; 00000001b u TRISB
```

```

    movwf    TRISB      ; RB0/INT kao ulaz, a ostali kao izlaz
    bcf      STATUS, RP0 ; Povratak u BANK0

; ***** Čitanje iz EEPROM memorije *****

    movlw    00h          ; Želimo pročitati sadržaj EEPROM
    movwf    EEADR        ; memorije sa adrese 00h
    bsf      STATUS, RP0  ; Prebacuje nas u BANK1
    bsf      EECON1, RD    ; Inicira proces čitanja
                        ; EEPROM memorije
    bcf      STATUS, RP0  ; Prebacuje nas u BANK0
    movf     EEDATA, W     ; Sadržaj bajta sa adrese 00h iz EEPROM
                        ; memorije je u EEDATA, pa u W
    bcf      STATUS, RP0  ; Povratak u BANK0

; ***** Inicijalizacija LED displeja i brojača *****

    movwf    DISP        ; Inicijalizacija brojača
    movwf    PORTB       ; Prikaz cifre na displeju

; ***** Snimanje u EEPROM *****

Gla    movf     DISP, W      ; DISP u W
        movwf    EEDATA     ; pa u EEDATA
        movlw    00h        ; na adresu
        movwf    EEADR      ; 00h
        bsf      STATUS, RP0 ; Prebacuje nas u BANK1 zbog EECON1
        bsf      EECON1, WREN ; Dozvola pisanja u EEPROM
        movlw    55h        ;
        movwf    EECON2     ;
        movlw    AAh        ; Inicijalizacija upisa u EEPROM
        movwf    EECON2     ;
        bsf      EECON1, WR  ;

Kraj   btfs    EECON1, WR    ; Test završetka snimanja
        goto     Kraj        ;

                        ; U ovom trenutku nastaje interapt

        bcf      EECON1, WREN ; Zabrana pisanja u EEPROM
        bcf      STATUS, RP0  ; Povratak u BANK0
        goto     Gla         ; Povratak na početak programa

; ***** Tabela *****

Tabl   movf     DISP, W      ; U promenljivoj DISP nalazi se vrednost od
        addwf    PCL, 1      ; 00h do 09h. Ta vrednost se dodaje na PCL.
        retlw    01111110b ; Obrazac za crtanje cifre 0 - 0
        retlw    00001100b ; Obrazac za crtanje cifre 1 - 1
        retlw    10110110b ; Obrazac za crtanje cifre 2 - 2

```

```

retlw    10011110b ; Obrazac za crtanje cifre 3 - 3
retlw    11001100b ; Obrazac za crtanje cifre 4 - 4
retlw    11011010b ; Obrazac za crtanje cifre 5 - 5
retlw    11111010b ; Obrazac za crtanje cifre 6 - 6
retlw    00001110b ; Obrazac za crtanje cifre 7 - 7
retlw    11111110b ; Obrazac za crtanje cifre 8 - 8
retlw    11011110b ; Obrazac za crtanje cifre 9 - 9

```

; ***** Kraj programa *****

end ; Kraj programa

Prilikom snimanja koda u PIC16F84 morate sadržaj memorijske adrese 00h u EEPROM memoriji podesiti na proizvoljan broj između 00h i 09h. U protivnom će mikrokontroler uzeti inicijalno stanje iz EEPROM memorije (FFh) i pri nailasku na tabelu izvršavanje programa će se nastaviti za 245 bajtova iza same tabele, što će najverovatnije izazvati blokadu mikrokontrolera. To je u programu učinjeno upotrebom DE pseudoinstrukcije. Njom se jedan (ili više) bajtova smeštaju u EEPROM. Budući da je za nju potrebno prethodno definisati početak upisa u EEPROM memoriju (org pseudoinstrukcijom i adresama od 2100h do 213Fh), najpraktičnije je staviti je ispred glavnog programa, kako se ne bi desilo da se i program greškom upiše u EEPROM.

Pri smeštanju više bajtova u EEPROM, možete bajt po bajt iza DE pseudoinstrukcije odvojiti zarezom, ovako: de 05h, A8h, 79h...

Iako izgleda da je EEPROM na neki način produžetak programske memorije, razlika svakako postoji. U EEPROM memoriji nije moguće izvršavati programe, isto kao što se ni iz samog programa ne može čitati (a ni menjati) sadržaj programske memorije.

Možda ste primetili da u rutini za snimanje u EEPROM nije izvršena zabrana korišćenja svih interapta resetovanjem GIE bita. Ukoliko bi se interapti zabranili, program nikada ne bi skočio na interapt rutinu, i neprestano bi se vrteo u petlji snimajući jedan isti bajt u EEPROM adresu 00h.

Primećujete da je tabela van interapt rutine iako se iz nje poziva. Pošto je ona napravljena kao podprogram, možete je postaviti bilo gde (ukoliko se izvršavanje programa ne nastavlja kroz nju).

Budući da ćete se u idućem poglavlju upoznati sa uspavanim stanjem mikrokontrolera, vreme je da naučite jednu lenju instrukciju. To je NOP (eng. **N**o **O**peration). Ova instrukcija, osim što traje jedan instrukcijski ciklus ne radi ništa korisno. Kada program mikrokontrolera naiđe na nju, utrošiće jedan instrukcijski ciklus nizašta. Preći će preko nje, kao da je nema. Uglavnom se koristi u situacijama kada je potrebno realizovati tačno određeno, ili kratko kašnjenje (npr. u petljama kašnjenja ili debouncing rutini).

Do sada ste naučili 33 od 35 instrukcija, potpunu upotrebu 14 registra i delimičnu upotrebu jednog (STATUS) od ukupno 15 registra.

22. Sleep mod - pojam

Pretpostavimo da želite napraviti hidroelektranu. Da biste proverili da li reka ima dovoljan protok vode tokom cele godine, odlučili ste da merite njen nivo mikrokontrolerom, i da prikupljene podatke povremeno (po popuni EEPROM memorije) prebacujete u laptop. Kako na reci nema električne mreže, mikrokontroler se napaja iz akumulatora. Negativni pol akumulatora (masa) spojen je sa rekom, a očitavanje nivoa vode se vrši interaptom pri promeni stanja na PORTB registru, bitovima 4 do 7, čiji su pinovi povezani sa odgovarajućim sondama na po 5cm dubine. Kada je nivo vode ispod najniže sonde, na svim pinovima je logička 1 (uključeni su interni otpornici na PORTB registru). Kada je nivo vode iznad najvišlje, voda se ponaša kao provodnik između mase i sonde, tako da je na svim pinovima logička 0. Mikrokontroler je povezan sa digitalnim časovnikom i dodatnom EEPROM memorijom, koji mu omogućavaju memorisanje tačnog trenutka rasta ili opadanja nivoa vode.

Očigledno je da u aktivnom stanju mikrokontroler radi samo povremeno (u trenutku povećanja ili smanjenja nivoa vode), i veoma kratko (vreme potrebno za čitanje tačnog vremena iz sata, i upis vremena i nivoa vode u EEPROM memoriju). Kako će tokom dana najverovatnije nastupiti jedan period plime i oseke, to iznosi samo oko 8 merenja dnevno. Preostalo vreme mikrokontroler će se vrteti u praznoj petlji čekajući na interapt, i bespotrebno trošeći struju iz akumulatora.

SLEEP „spavajući“ mod predstavlja stanje u kojem se mikrokontroler nalazi u režimu smanjene potrošnje energije. U SLEEP modu blokira se rad glavnog oscilatora. Instrukcije se prestaju izvršavati, tajmer prestaje sa radom, a WDT se (ukoliko je uključen) resetuje (ali nastavlja sa radom zbog sopstvenog oscilatora). Portovi zadržavaju svoja stanja (izlaz – 0, izlaz – 1 ili ulaz – stanje visoke impedanse). Za najmanju moguću potrošnju potrebno je projektovati spoljna elektronska kola tako da ne vuku struju iz izlaznih pinova portova, i da ulazne portove postave na stabilan logički nivo (0 ili 1). T0CKL pin bi takođe trebao biti u stabilnom nivou.

Napon napajanja mikrokontrolera može se spustiti i do 1,5V bez gubitka podataka. Potrošnja mikrokontrolera pada sa oko 2mA na oko 5μA.

Ukoliko Vam je potrošnja struje izuzetno bitna, možete kupiti verziju mikrokontrolera specijalno napravljenu za rad sa što manjom potrošnjom struje.

U SLEEP mod ulazi se SLEEP instrukcijom, a iz njega se mikrokontroler može „probuditi“ na tri načina.

1. Dovođenjem logičke nule na $\overline{\text{MCLR}}$ pin mikrokontrolera, što prouzrokuje reset mikrokontrolera. Više o resetu, u idućim poglavljima.
2. Resetom preko isteka vremena WDT.
3. Interaptom na RB0/INT pinu, interaptom pri promeni stanja na PORTB registru, bitovima 4 do 7, ili interaptom izazvanim završetkom upisa u EEPROM.

Interapt izazvan tajmerom ne može se koristiti za buđenje iz SLEEP moda, jer je u SLEEP modu i tajmer uspavan.

Ulaskom u SLEEP mod, menja se stanje određenih bitova STATUS registra. Vreme je da napokon proučite i njegovu unutrašnju strukturu.

bit7							bit0
/	/	RP0	$\overline{\text{TO}}$	$\overline{\text{PD}}$	Z	DC	C

- Bitovi 7 i 6 ne koriste se u PIC16F84. Trebaju biti u stanju logičke 0.
- Bit 5 (RP0) koristi se kod direktnog adresiranja za izbor banke podataka po sledećem :
0 - BANK0 – 00h-7Fh
1 - BANK1 – 80h-FFh
- $\overline{\text{TO}}$ (eng. **T**ime-**O**ut) „prekoračenje vremena“ bit je na stanju logičke 0 nakon isteka WDT, a u stanju logičke 1 po dolasku napona napajanja, i izvršenju CLRWDT ili SLEEP instrukcije. Na osnovu stanja ovog bita program može detektovati da je reset mikrokontrolera nastao usled prekoračenja WDT.
- $\overline{\text{PD}}$ (eng. **P**ower-**D**own) bit postavlja se na 1 po dolasku napona napajanja ili izvršenju CLRWDT instrukcije, a na 0 po izvršenju SLEEP instrukcije.
- Logička jedinica na Zero “nultom” bitu pokazuje da je rezultat zadnje aritmetičke ili logičke operacije jednak 0. Logička nula na ovom bitu ukazuje da je rezultat različit od 0.
- DC (eng. **D**igit **C**arry/borrow) bit „bit prekoračenja/pozajmice“ indikuje prekoračenje ili pozajmicu donjem niblu nastalu zbog izvršavanja instrukcija sabiranja ili oduzimanja.
- Logička jedinica na C (eng. **C**arry/borrow) bitu „bitu prekoračenja“ ukazuje da je došlo do prekoračenja ili pozajmice u bajtu nad kojim se izvršila aritmetička, logička ili operacija rotacije.

Kao što vidite iz samog STATUS registra, izvršenje SLEEP instrukcije prouzrokuje resetovanja $\overline{\text{PD}}$ bita i setovanje $\overline{\text{TO}}$ bita.

Možda vam čudno izgleda buđenje iz SLEEP moda istekom WDT. On se u mikrokontroleru koristi za povremeno buđenje iz SLEEP moda, obavljanje određene operacije, i ponovni ulazak u SLEEP mod. Tako mikrokontroler može povremeno izvršiti poneku operaciju, a opet će većinu vremena provesti spavajući (i trošeći manje struje). Kako istek WDT prouzrokuje softverski reset mikrokontrolera program mora po resetu testirati $\overline{\text{TO}}$ bit, i na osnovu njegovog stanja zaključiti da li se radi o prvom uključenju mikrokontrolera, ili o prekidu nastalom istekom WDT, kako bi se mogao nastaviti započeti program.

U praksi se mnogo češće sreće buđenje interaptima. U sledećem poglavlju upoznaćete se sa specifičnostima upotrebe interapta u SLEEP modu.

23. Buđenje iz Sleep moda

Buđenje iz SLEEP moda interaptima razlikuje se od običnih interapta po tome što je moguće da se umesto skoka na interapt rutinu, izvršavanje programa nastavi iza SLEEP instrukcije. Naime, ukoliko je u SLEEP modu GIE setovan, pri interaptu će se mikrokontroler probuditi i skočiti na interapt rutinu, kao i u običnom programu. Ali, ukoliko je GIE resetovan, pri interaptu (dozvoljenom odgovarajućom interapt dozvolom) će se probuditi i nastaviti dalje izvršenje programa, bez skoka na interapt rutinu.

U zavisnosti od trenutka sticanja uslova za pojavu interapta moguće je da se interapt dogodi neposredno pre SLEEP instrukcije. U tom slučaju, SLEEP instrukcija se neće izvršiti, pa neće biti resetovan WDT i njegov preskaler, resetovan PD bit i setovan TO bit. Zbog toga je potrebno ručno resetovati WDT (ukoliko se koristi) neposredno pre SLEEP instrukcije. U slučaju da je potrebno saznati da li se izvršila SLEEP instrukcija, to se može proveriti testiranjem PD bita STATUS registra.

Sada ćete se upoznati sa programom koji ilustruje upotrebu SLEEP moda i buđenje iz njega na oba načina (sa setovanim i resetovanim GIE). Program se sastoji iz dva dela. U prvom delu, program ulazi u SLEEP mod, i od korisnika se zahteva da postavi kombinaciju tastera T1 i T2 i onda kratko pritisne T3 izazivajući interapt. T1, T2 kombinacija snima se u memoriju mikrokontrolera iz interapt rutine primenom indirektnog adresiranja. Postupak se ponavlja sve dok se ne snimi 5 kombinacija. U drugom delu, mikrokontroler se ponovo uspavljuje ali tako da interapt prouzrokuje njegovo buđenje i nastavak programa bez skoka na interapt rutinu. Po izazivanju interapta (T3) snimljene kombinacije se čitaju i prikazuju na LED1 i LED2, Nakon 5 prikazanih kombinacija, program se ponavlja iz početka.

; ***** Inicijalizacija i imenovanje *****

```
list      p=16F84      ; Definiše upotrebljeni mikrokontroler
#include   <p16F84.inc>  ; Ubacuje nazive registra u program
__CONFIG  _CP_OFF & _WDT_OFF & _PWRTE_ON & _RC_OSC
                        ; Podešava konfiguracione bitove
TEMPW     equ    0Ch    ; Promenljiva za W registar
TEMPSTATUS equ    0Dh    ; Promenljiva za STATUS registar
```

; 0Eh – 12h memorija za snimanje stanja prekidača

; Vrednost 13h koristi se u programu za test prekoračenja memorije

; ***** Interapt rutina za prvi deo programa – normalne interapte *****

```
org       00h          ; Ovde PC dolazi pri uključenju i resetu
goto      Main         ; Odlazak na glavni program
org       04h          ; Ovde će početi interapt rutina
movwf     TEMPW         ; Čuvanje sadržaja W registra u TEMPW
swapf     STATUS,W      ; STATUS sa okrenutim niblovima u W
movwf     TEMPSTATUS    ; i zatim u TEMPSTATUS registar
```

```

rlf          PORTA, W      ; Uzmi stanje prekidača sa PORTA registra,
movwf        INDF          ; rotiraj ga ulevo i snimi ga u memoriju
incf         FSR           ; Pozicioniranje pointera na sledeći registar

swapf        TEMPSTATUS, W ; TEMPSTATUS sa okrenutim niblovima
                                ; u W. Dva puta okrenuti niblovi daju
movwf        STATUS        ; prvobitno stanje koje ide u STATUS
swapf        TEMPW, F      ; Jednom okreni niblove u samom
                                ; TEMPW registru,
swapf        TEMPW, W      ; a drugi put sa W kao odredištem.
bcf          INTCON, INTF   ; Obriši interapt flag - dozvoli
                                ; nove interapte
retfie       ; Kraj interapt rutine.

```

; ***** Glavni program – inicijalizacija *****

```

Main bsf       STATUS, RP0    ; Prebacuje nas u BANK1

movlw        01h            ; 00000001b u TRISB
movwf        TRISB          ; RB0/INT kao ulaz, a ostali kao izlaz
movlw        03h            ; 00000011b u TRISA
movwf        TRISA          ; Prekidači T1 i T2 kao ulaz, a ostatak kao izlaz

movlw        00000000b      ; INTEDG=0 - Interapt se izaziva pri
                                ; opadajućoj ivici signala (sa 1 na 0)
                                ; NOT_RBPU=0 - Uključeni interni „pull up“
movwf        OPTION_REG     ; otpornici na PORTB

movlw        0Eh            ; Inicijalizacija pointera RAM memorije
movwf        FSR            ; na registar 0Eh

movlw        10010000b      ; bit 7 GIE – Global interrupt enable (1 – uključi)
                                ; bit 4 INTE – RB0 Interrupt enable (1 – uključi)
movwf        INTCON         ; bit 1 INTF – Interapt flag (0=obriši)

bcf          STATUS, RP0    ; Povratak u BANK0

```

; ***** Prvi deo programa – setovan GIE, normalni interapti *****

```

movlw        00001100b      ; Obrazac za crtanje cifre 1 - 1
movwf        PORTB          ; prikaži tekući deo programa na LED displeju.
Spa sleep      ; Uspavaj mikrokontroler do pojave interapta.
                                ; Po njegovoj pojavi, skoči na interapt rutinu,
                                ; i po izlasku iz nje, vrati se ovde.
movlw        EDh            ; Kraj memorije 13h + EDh = setovan Carry flag
addwf        FSR, W          ; Saberi sa vrednošću iz pointera, rezultat u W
btfs         STATUS, C      ; Da li je došlo do prekoračenja?
goto         Spa            ; Nije, nastavi sa uzimanjem stanja.
Zabr bcf       INTCON, GIE    ; Jeste. Zabrani nove interapte

```

```

btfsf      INTCON, GIE      ; Proveri jesu li interapti zaista zabranjeni
goto  Zabr                  ; Nisu, zabrani ih opet.
; Jesu, nastavi sa drugim delom programa.

; ***** Drugi deo programa – resetovan GIE, interaptima se mikrokontroler *****
; ***** budi i nastavlja izvršavanje programa *****

movlw      10110110b ; Obrazac za crtanje cifre 2 - 2
movwf      PORTB      ; prikaži tekući deo programa na LED displeju.

movlw      0Eh          ; Inicijalizacija pointera RAM memorije
movwf      FSR          ; na registar 0Eh
Spa2 bcf      INTCON, INTF      ; Obriši interapt flag - dozvoli
                                   ; nove interapte
sleep                                ; Uspavaj mikrokontroler do pojave interapta
                                   ; pri čemu se pri buđenju ne skače na
                                   ; interapt rutinu, već se izvršava sledeća (nop)
                                   ; instrukcija
nop                                ; Utroši jedan instrukcijski ciklus tek radi
                                   ; ilustracije upotrebe NOP instrukcije
rlf        INDF, W        ; Uzmi sadržaj iz memorije, rotiraj ga ulevo,
movwf      PORTA        ; i prikaži ga na LED1 i LED2.
                                   ; Duplom rotacijom (pri snimanju i čitanju)
                                   ; su stanja prekidača T1 i T2 došla na mesto
                                   ; prikaza rezultata LED1 i LED2.
incf      FSR            ; Pozicioniranje pointera na sledeći registar
movlw      EDh            ; Kraj memorije 13h + EDh = setovan Carry flag
addwf      FSR, W        ; Saberi sa vrednošću iz pointera, rezultat u W
btfsf      STATUS, C      ; Da li je došlo do prekoračenja?
goto      Spa2            ; Nije, nastavi sa prikazom stanja
goto      Main            ; Jeste, počni program od početka

; ***** Kraj programa *****

end                                ; Kraj programa

```

Zabr rutina služi za sigurno resetovanje GIE bita (setite se napomena uz interapte).

Sleep mod ima jedan nedostatak, koji se ogleda u nestabilnosti oscilatora prilikom buđenja. Naime, pošto je oscilator za vreme SLEEP moda blokiran, nije mu baš lako da odjednom počne proizvoditi čiste taktove. Buđenje neće biti moguće dok se oscilator ne stabilizuje. Zato se pri korišćenju SLEEP moda ne može vršiti merenje vremena brojanjem instrukcijskih ciklusa instrukcija u programu, čak iako se buđenje interaptom dešava tačno određeno vreme nakon ulaska u Sleep mod.

24. Reset

Najjednostavnije rečeno, reset služi da izvršavanje programa počne iz početka.

PIC16F84 ima 3 izvora reseta. Prvi je svakako reset prilikom dovođenja napona napajanja (eng. POR – **P**ower-**o**n **R**eset). Unutar mikrokontrolera nalazi se malo električno kolo koje detektuje porast napona napajanja do napona dovoljnog za rad mikrokontrolera. U trenutku dostizanja nominalnog napona, mikrokontroler se resetuje. Ovim se sprečava rad mikrokontrolera pri preniskom naponu napajanja. Ukoliko je potrebno, može se uključiti PWRTE konfiguracioni bit koji drži mikrokontroler u stanju reseta oko 72mS od dovođenja napona napajanja. To je dovoljno da se napon napajanja podigne na nominalnu vrednost i da oscilator mikrokontrolera počne proizvoditi čiste neprigušene oscilacije.

Drugi izvor reseta je MCLR pin mikrokontrolera. On mora uvek biti držan na logičkoj jedinici. Obično se to realizuje povezivanjem sa naponom napajanja preko otpornika od oko 100k Ω . Pri dovođenju logičke 0 na ovaj pin (kratko spajanje sa masom), mikrokontroler se resetuje. Za svaki slučaj u ulaznom stepenu ovog pina postavljen je Šmitov okidač koji ima ulogu filtriranja slabih impulsa, koji bi mogli prouzrokovati nehotični reset mikrokontrolera.

Zadnji izvor reseta naučili ste u poglavlju sa WDT. U slučaju potrebe, može se izvršiti softverski reset, namernim izostavljanjem CLRWDT instrukcije u željenom delu programa.

Nastavak sledi

Molio bih prodavce PIC mikrokontrolera da mi se jave radi objavljivanja njihovih kontakt podataka (adresa, tel, mob, mail, sajt, baner...) u ovom uputstvu po njegovom završetku. Zbog zaštite od spama, e-mail će biti prikazan u obliku slike.

U uputstvu ima izvesnih nedorečenosti u pogledu funkcija pojedinih registara, tajminga i sličnih stvari. Kako je ovo prvenstveno početničko uputstvo, ne smatram da je bitno opterećivati čitaoca znanjem koje mu verovatno nikada neće ni koristiti. Za profesionalan rad sa PIC mikrokontrolerima, naučite engleski i čitajte Microchipova tehnička uputstva (Datasheet).

Primitili ste grešku? Prvo proverite da li je ispravljena u najnovijem uputstvu (na sajtu kao i u uputstvu uvek objavljujem i datum izlaska nove verzije), i ukoliko nije kontaktirajte me preko knjige utisaka ili na mail objavljen na sajtu.

Uputstvo se može slobodno kopirati i distribuirati pod uslovom da se u njega ne unose nikakve izmene. Delimično kopiranje uputstva je dozvoljeno uz navođenje linka do mog sajta, ali nije preporučljivo zbog novih verzija uputstva. Međutim, **prodaja ovog uputstva je zabranjena**. Ukoliko ste ovo uputstvo kupili bilo gde i u bilo kom obliku (na CD-u na buvljaku, na floppy od kolege sa posla – pirata, od lokalnog TV servisera u štampanom izdanju), molim Vas da me o tome obavestite.

Autor ne snosi nikakvu odgovornost za korišćenje ovog uputstva. Ako spržite PIC, gomilu elektronskih komponenti ili najnoviji Macintosh laptop, kupite novi, ali o svom trošku.

Uputstvo postavljeno 6.5.2006. na sajtu
<http://www.ptt.yu/korisnici/t/r/trifunov/index.htm>